TRABAJO DE GRADO

SISTEMA MULTI-AGENTE PARA LA EVALUACIÓN ESTRATÉGICA DE LICITACIONES PÚBLICAS EN FÁBRICAS DE SOFTWARE COLOMBIANAS

DAVID LARA SANCHEZ 67001029

DLARA29@UCATOLICA.EDU.CO

JUAN DAVID LOPEZ RODRIGUEZ 67000973

JDLOPEZ73@UCATOLICA.EDU.CO

UNIVERSIDAD CATÓLICA DE COLOMBIA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
BOGOTÁ D.C
2026

TRABAJO DE GRADO

SISTEMA MULTI-AGENTE PARA LA EVALUACIÓN ESTRATÉGICA DE LICITACIONES PÚBLICAS EN FÁBRICAS DE SOFTWARE COLOMBIANAS

DAVID LARA SANCHEZ 67001029

JUAN DAVID LOPEZ RODRIGUEZ 67000973

Trabajo de grado presentado para optar al título de
Ingeniero de Sistemas y Computación

Docente
SOLANGIE PAOLA GARAVITO MENDIVELSO
Magíster en Gerencia de la Innovación Empresarial
spgaravito@ucatolica.edu.co

UNIVERSIDAD CATÓLICA DE COLOMBIA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN
BOGOTÁ D.C
2026

Nota de aceptación:

_____
_____
_____
_____
_____

_____
Firma del presidente del jurado

_____
Firma del jurado

_____
Firma del presidente del jurado

Ciudad y fecha (día, mes, año)

Dedicatoria (opcional)

Se relacionan personas cercanas
Texto centrado inferior a media página. Tamaño 10 puntos

Agradecimientos (opcional)

Se relacionan terceros

CONTENIDO

INTRODUCCIÓN 13
1. GENERALIDADES 14
1.1 CONTEXTO DEL PROBLEMA 15
1.2 PLANTEAMIENTO DEL PROBLEMA 16
1.3 JUSTIFICACIÓN 16
1.4 OBJETIVOS 18
1.4.1 Objetivo General. 18
1.4.2 Objetivos Específicos 18
1.5.1 Marco Conceptual. 19
1.5.3 Marco Jurídico 21
1.4 DISEÑO METODOLÓGICO 21
2. DEFINICIÓN DE REQUISITOS DEL SISTEMA MULTI-AGENTE 22
2.1 MÉTODO Y CRITERIOS DE ESPECIFICACIÓN 23
2.2 ÉPICAS DEL SISTEMA 24
2.3 HISTORIAS DE USUARIO Y CRITERIOS DE ACEPTACIÓN 25
2.4 REQUISITOS FUNCIONALES ARQUITECTÓNICAMENTE SIGNIFICATIVOS 27
2.5 REQUISITOS NO FUNCIONALES ARQUITECTÓNICAMENTE SIGNIFICATIVOS 28
2.6 RESULTADO DEL OBJETIVO Y ARTICULACIÓN CON EL SEGUNDO OBJETIVO 29
3. DISEÑO DE LA ARQUITECTURA DE SOFTWARE DEL SISTEMA MULTI-AGENTE 30
3.1 MARCO DE DESCRIPCIÓN ARQUITECTÓNICA SEGÚN ISO/IEC/IEEE 42010:2022 31
3.2 PARTES INTERESADAS, PERSPECTIVAS Y PREOCUPACIONES 32
3.3 IMPULSORES ARQUITECTÓNICOS: ÁRBOL DE UTILIDAD, RESTRICCIONES Y FUNCIONES CRÍTICAS 33
3.4 SELECCIÓN DEL ESTILO ARQUITECTÓNICO 34
3.5 PATRÓN DE COORDINACIÓN DE LOS AGENTES 37
3.6 DECISIONES ARQUITECTÓNICAS Y SU JUSTIFICACIÓN 38
3.7 CONSIDERACIONES EVALUADAS QUE NO APLICAN AL MVP 40
3.8 PUNTOS DE VISTA Y HERRAMIENTA DE MODELADO 41
3.9 VISTA DE CONTEXTO (MODELO C4, NIVEL 1) 42
3.10 VISTA DE CONTENEDORES (MODELO C4, NIVEL 2) 43
3.11 VISTAS DE COMPONENTES (MODELO C4, NIVEL 3) 44
3.12 VISTA DINÁMICA DEL FLUJO DE ANÁLISIS 46
3.13 VISTA DE DESPLIEGUE 46
3.14 CORRESPONDENCIAS Y TRAZABILIDAD 47
3.15 RESULTADO PARCIAL DEL OBJETIVO Y PENDIENTES DECLARADOS 48

6. CONCLUSIONES 50
7. RECOMENDACIONES 50
BIBLIOGRAFÍA 51
ANEXOS 55

LISTA DE TABLAS

pág.

Tabla 1. Resumen de la línea base de requisitos 24
Tabla 2. Catálogo de épicas del sistema multi-agente 26
Tabla 3. Historias de usuario agrupadas por épica 27
Tabla 4. Requisitos funcionales arquitectónicamente significativos 28
Tabla 5. Requisitos no funcionales arquitectónicamente significativos 30
Tabla 6. Correspondencia entre la línea base de requisitos y los insumos del segundo objetivo 31
Tabla 7. Instanciación de los conceptos de ISO/IEC/IEEE 42010:2022 en el proyecto 32
Tabla 8. Partes interesadas agrupadas por perspectiva 33
Tabla 9. Preocupaciones arquitectónicas y escenarios que las cuantifican 34
Tabla 10. Restricciones y funciones críticas que condicionan la arquitectura 35
Tabla 11. Evaluación de estilos arquitectónicos frente a los impulsores 36
Tabla 12. Patrones de coordinación de agentes evaluados 39
Tabla 13. Registro de decisiones arquitectónicas 40
Tabla 14. Contrapartidas de las decisiones arquitectónicas 41
Tabla 15. Evaluación de marcos de interfaz web en Python 42
Tabla 16. Consideraciones evaluadas que no aplican al MVP 43
Tabla 17. Puntos de vista adoptados 44
Tabla 18. Contenedores del sistema y decisiones que los justifican 47
Tabla 19. Correspondencia entre escenarios ASR, decisiones, elementos y vistas 51

LISTA DE FIGURAS

pág.

Figura 1. Árbol de utilidad del sistema multi-agente 35
Figura 2. Vista de contexto del sistema multi-agente (C4 Nivel 1)..... 46
Figura 3. Vista de contenedores del sistema multi-agente (C4 Nivel 2) 47
Figura 4. Componentes de la API de aplicación (C4 Nivel 3) 48
Figura 5. Componentes del trabajador de análisis multi-agente (C4 Nivel 3)..... 49
Figura 6. Flujo de análisis con revisión humana en el ciclo (vista dinámica)..... 49
Figura 7. Despliegue de referencia del MVP en Azure..... 50

LISTA DE ANEXOS

pág.

Anexo A. Matriz de requisitos del sistema multi-agente 57
Anexo B. Matriz ASR y árbol de utilidad 57
Anexo C. Diagramas de arquitectura en formato ampliado 57
Anexo D. Código fuente de los diagramas de arquitectura 65

GLOSARIO

- ADECUACIÓN FUNCIONAL: Capacidad del producto de software para proporcionar funciones que satisfacen de manera exacta, completa y pertinente las necesidades declaradas e implícitas de los usuarios bajo condiciones especificadas, según el modelo de calidad NTC-ISO/IEC 25010.
- ÁRBOL DE UTILIDAD (UTILITY TREE): Representación jerárquica que descompone la utilidad de un sistema en atributos de calidad, refinamientos y escenarios medibles, y los prioriza según su valor de negocio y su impacto en la arquitectura.
- ARQUITECTURA DE SOFTWARE: Conceptos y propiedades fundamentales de un sistema en su ambiente, que definen su estructura, componentes y los principios que gobiernan su realización, evolución y diseño a lo largo del tiempo, tal como lo establece la norma ISO/IEC/IEEE 42010.
- ASEGURAMIENTO DE CALIDAD (QA): Conjunto de procesos de enfoque preventivo y proactivo que se implementan durante todo el ciclo de vida del desarrollo de software, con el objetivo principal de evitar que los defectos o vulnerabilidades se introduzcan y lleguen al producto final.
- ASR (REQUISITO ARQUITECTÓNICAMENTE SIGNIFICATIVO): Requisito que tiene un efecto medible sobre la estructura del sistema, de modo que su cambio tardío obligaría a rediseñar y no solo a modificar un componente.
- CONTROL DE CALIDAD (QC): Procedimiento de enfoque correctivo a través del cual se examina y confirma que los productos de software ya desarrollados o terminados satisfacen los requisitos definidos, focalizándose en detectar, registrar y solucionar fallas o bugs antes de su liberación.
- DEVSECOPS: Marco de trabajo que fomenta la colaboración y la responsabilidad compartida integrando la seguridad de la información (Sec) en todas las fases de desarrollo (Dev) y operaciones (Ops), permitiendo automatizar pruebas continuas en el ciclo de vida del software.
- EFICIENCIA DE DESEMPEÑO: Característica del modelo de calidad de software que evalúa el comportamiento temporal de un sistema, sus tiempos de respuesta y la correcta utilización de recursos informáticos bajo diversas condiciones de carga.
- ÉPICA: Unidad de trabajo de gran tamaño que agrupa historias de usuario relacionadas con una misma capacidad o etapa del proceso de negocio.
- HISTORIA DE USUARIO: Descripción breve de una funcionalidad desde la perspectiva de quien la usa, redactada con la estructura Como–Quiero–Para y acompañada de criterios de aceptación verificables.
- IA AGÉNTICA (INTELIGENCIA ARTIFICIAL MULTI-AGENTE): Rama avanzada de la inteligencia artificial fundamentada en sistemas autónomos (agentes) capaces de comprender su entorno, tomar decisiones, establecer objetivos, planificar y ejecutar estrategias complejas o colaborativas con mínima supervisión humana.
- MODELO C4: Herramienta metodológica y estándar visual para la descripción de la arquitectura de software. Desglosa los sistemas jerárquicamente en cuatro niveles de abstracción (Contexto, Contenedores, Componentes y Código) para facilitar la comprensión técnica y la escalabilidad.
- MODELO DE LENGUAJE DE GRAN ESCALA (LLM): Modelo de inteligencia artificial entrenado con grandes volúmenes de texto que genera, resume o clasifica lenguaje

natural a partir de una instrucción.

- PRODUCTO MÍNIMO VIABLE (MVP): Versión de un producto con las funcionalidades mínimas necesarias para validar su propuesta de valor con usuarios reales.
- REQUISITOS FUNCIONALES (RF): Declaraciones técnicas que especifican qué es exactamente lo que el sistema de software debe hacer. Incluyen el detalle de las funciones, las reglas de negocio, y cómo debe reaccionar el sistema ante determinadas entradas de datos.
- REQUISITOS NO FUNCIONALES (RNF): Especificaciones técnicas que describen cómo debe comportarse el sistema. Imponen restricciones y definen los atributos de calidad requeridos para el software, tales como el rendimiento, la mantenibilidad, la seguridad y la usabilidad.
- SCRUM: Marco de trabajo de metodología ágil diseñado para gestionar el desarrollo de productos complejos mediante entregas iterativas e incrementales en ciclos cortos de tiempo denominados *Sprints*, fomentando la adaptación rápida al cambio y la mejora continua.
- SECOP II: Sistema Electrónico de Contratación Pública administrado por la Agencia Nacional de Contratación Pública – Colombia Compra Eficiente, en el que las entidades estatales publican y gestionan sus procesos de contratación.
- SHIFT-LEFT: Enfoque técnico y metodológico en el desarrollo de software que consiste en "desplazar a la izquierda" (hacia las etapas más tempranas del ciclo de vida) las pruebas de calidad, validación y seguridad, previniendo defectos antes de que lleguen a etapas tardías de producción.
- STAKEHOLDER (PARTE INTERESADA): Rol, posición, individuo u organización que tiene un interés, derecho o preocupación de carácter relevante frente al desarrollo, operación o resultados de un sistema de software.
- TRAZABILIDAD: Capacidad de rastrear el ciclo de vida completo de un requisito, desde su origen y conceptualización, pasando por el diseño y la codificación, hasta su validación a través de los casos de prueba ejecutados en el sistema.

RESUMEN

Las fábricas de software del sector de Tecnologías de la Información en Colombia dedican jornadas completas al análisis manual de pliegos de condiciones en licitaciones públicas, lo que deriva en la pérdida de convocatorias o en la postulación a proyectos inviables. El problema tiene un peso económico concreto: el mercado estatal supera los $174,8 billones de pesos anuales, pero convertir ese volumen de información en decisiones acertadas sobre en qué contratos participar, cómo reducir sobrecostos y cómo evitar riesgos sigue dependiendo exclusivamente de la lectura humana.

Como respuesta, este proyecto desarrolla un Producto Mínimo Viable (MVP): un producto de software basado en un sistema de Inteligencia Artificial multi-agente, concebido para ser utilizado por cualquier fábrica de software y no para una empresa en particular. El sistema automatiza la extracción de requisitos de las licitaciones y evalúa el nivel de cumplimiento de la organización, cruzándolos con su experiencia, portafolio y capacidades operativas. Su construcción sigue el marco ágil Scrum adaptado con prácticas DevSecOps, bajo los estándares ISO/IEC/IEEE 29148:2018 para requisitos, ISO/IEC/IEEE 42010:2022 con Modelo C4 para arquitectura y NTC-ISO/IEC 25010 para la evaluación de calidad (QA/QC).

El resultado es un prototipo funcional capaz de procesar múltiples oportunidades y generar un reporte estratégico que detalla el porcentaje de cumplimiento técnico por cada licitación. Esta automatización permite emitir recomendaciones de viabilidad precisas, reduciendo el error humano y liberando al equipo comercial para que se concentre en el cierre de negocios.

PALABRAS CLAVE: Administración pública, Control de calidad, Inteligencia artificial, Software de computadora, Toma de decisiones.

ABSTRACT

Software development companies in Colombia's Information Technology sector dedicate entire days to manually analyzing tender specifications, resulting in missed opportunities or applications for unfeasible projects. This problem has a significant economic impact: the public sector market exceeds 174.8 trillion Colombian pesos annually yet converting this vast amount of information into sound decisions about which contracts to bid on, how to reduce cost overruns, and how to avoid risky ventures still relies on human interpretation.

In response, this project develops a Minimum Viable Product (MVP): a software product based on a multi-agent Artificial Intelligence system, designed for use by any software development company, not just a specific firm. The system automates the extraction of tender requirements and assesses the organization's level of compliance, cross-referencing them with its experience, portfolio, and operational capabilities. Its development follows the adapted Scrum agile framework with DevSecOps practices, adhering to ISO/IEC/IEEE 29148:2018 for requirements, ISO/IEC/IEEE 42010:2022 with Model C4 for architecture, and NTC-ISO/IEC 25010 for quality assurance (QA/QC).

The result is a functional prototype capable of processing multiple opportunities and issuing feasibility recommendations for each tender, reducing human error and freeing up the sales team to focus on closing deals.

KEYWORDS: Artificial intelligence, Computer software, Decision making, Public administration, Quality control.

INTRODUCCIÓN

En Colombia, la contratación pública se ha consolidado como uno de los principales motores de adquisición de servicios tecnológicos, alcanzando un mercado superior a los $174,8 billones de pesos anuales gestionados mediante plataformas transaccionales digitales como SECOP II y TVEC. Esta modernización gubernamental ha generado un volumen masivo de datos abiertos que, si bien promueve la transparencia, ha transformado radicalmente el ecosistema comercial para los proveedores del Estado. Las fábricas de software, en su afán por participar en este mercado, se enfrentan al reto de procesar diariamente cientos de pliegos de condiciones y anexos técnicos dispersos en estas plataformas. El origen de la presente propuesta surge de la necesidad de adaptar a estas organizaciones a una realidad digital donde el éxito comercial ya no depende únicamente de la capacidad técnica de desarrollo, sino de la agilidad tecnológica para filtrar grandes volúmenes de información e identificar las convocatorias que se alineen con su portafolio de servicios.

A pesar de esta digitalización estatal, persiste una problemática operativa crítica: las empresas continúan dependiendo del análisis humano y manual para evaluar la viabilidad de la extensa documentación legal y técnica, lo que incrementa los costos, ralentiza la toma de decisiones y eleva el riesgo de participar en licitaciones inviables. Para hacer frente a esta situación, el objetivo general de este proyecto es desarrollar un Producto Mínimo Viable (MVP) mediante un sistema de inteligencia artificial multi-agente que automatice la extracción de requisitos y evalúe el cumplimiento técnico de la organización. Desde su fundamentación teórica, esta solución se apoya en los lineamientos de la norma ISO/IEC/IEEE 29148:2018 para la ingeniería de requisitos y en el estándar ISO/IEC/IEEE 42010:2022 junto con el Modelo C4 para el diseño de la arquitectura. Metodológicamente, la construcción del prototipo se ejecuta bajo el marco de trabajo ágil Scrum, integrando prácticas DevSecOps para garantizar un desarrollo seguro, y finalizando con un plan de validación de calidad estructurado bajo la norma NTC-ISO/IEC 25010.

La aplicación de este ecosistema agéntico resulta de vital importancia para el área de la ingeniería de software y la gestión empresarial, ya que propone un cambio de paradigma en la forma en que se aborda la inteligencia comercial dentro de la industria tecnológica. Al proponer herramientas autónomas capaces de transformar datos crudos en análisis de viabilidad, se busca fortalecer la competitividad del sector frente a un mercado estatal altamente exigente. En este sentido, la investigación se enmarca en el siguiente planteamiento central: ¿De qué manera la implementación de un sistema multi-agente, ante la ineficiencia operativa generada por el análisis manual masivo de pliegos de condiciones, permite optimizar la evaluación estratégica de oportunidades de negocio en el mercado de contratación pública de Colombia para las fábricas de software?

1. GENERALIDADES

1.1 CONTEXTO DEL PROBLEMA

La revisión de antecedentes tecnológicos y metodológicos de este proyecto se estructura partiendo de un panorama global de herramientas y marcos teóricos, descendiendo hacia implementaciones regionales e internacionales específicas, para finalmente aterrizar en el contexto local colombiano y la brecha tecnológica que da origen al problema concreto de investigación.

A nivel mundial, el análisis de altos volúmenes de requisitos documentales ha impulsado la creación de plataformas de alcance global centradas en la inteligencia comercial y la gestión del ciclo de compras. En este ámbito, teóricos como Guida et al. (2023) presentan una revisión sistemática del estado del arte de la IA en *procurement* y proponen una agenda de investigación basada en las etapas del ciclo de adquisiciones a nivel global. A nivel industrial, destacan plataformas como Euna Procurement (ex-Bonfire), una solución SaaS en la nube que consolida la evaluación de propuestas e incorpora módulos de IA para identificar ahorros, orientada principalmente al sector público norteamericano. A diferencia del presente proyecto, esta herramienta está diseñada para el comprador institucional y no evalúa si un proveedor específico cumple los requisitos técnicos. Por otro lado, herramientas como Loopio utilizan IA generativa para extraer requisitos y generar borradores de respuestas a RFPs (Request for Proposal) para empresas tecnológicas internacionales. Aunque Loopio comparte la lógica de extracción, se enfoca en generar respuestas, careciendo de un análisis estratégico de viabilidad bajo una arquitectura multi-agente.

Al acotar el panorama hacia soluciones tecnológicas implementadas en marcos gubernamentales específicos, se observan esfuerzos por aplicar modelos de lenguaje y aprendizaje automático a pliegos de condiciones en Europa y Asia. En España, plataformas como LicitaBot y Tendios aplican procesamiento de lenguaje natural e IA para monitorear portales gubernamentales, detectar entregables requeridos y analizar históricos de ganadores, aunque no cruzan los requisitos del documento con las capacidades operativas de la organización. Desde la academia europea, Siciliani et al. (2023) proponen un Sistema de Soporte de Decisiones multi-módulo para el sistema público italiano, mientras que Nai, Sulis & Meo (2023) aplican *Machine Learning* para recomendar oportunidades basándose en patrones históricos, pero sin analizar la aptitud técnica actual de la empresa proveedora. En el contexto asiático y de los países nórdicos, Sofat & Sodhi (2024) construyeron un pipeline LLM+RAG en India para verificar automáticamente el cumplimiento de documentos de proveedores frente a licitaciones, y Waseem et al. (2024) diseñaron un asistente de IA para evaluación de licitaciones piloteado en una empresa de software finlandesa. Aunque estos desarrollos son metodológicamente cercanos a la presente propuesta, carecen de orquestación multi-agente y adaptación al contexto normativo colombiano.

En el ámbito nacional colombiano, las soluciones existentes se limitan a la publicación de datos abiertos o al monitoreo superficial, sin capacidades analíticas avanzadas. Existen agregadores comerciales como Licitaciones.com que emiten alertas por sector y tipo de proceso en Colombia y la región, limitándose a notificar la existencia de una licitación sin analizar estratégicamente la capacidad técnica de la empresa para ganarla. Paralelamente, el mercado estatal opera centralizado en SECOP II, la plataforma transaccional administrada por Colombia Compra Eficiente donde las entidades estatales publican sus procesos. Si bien sus datos están disponibles como conjuntos abiertos, el sistema actúa únicamente como repositorio, publicando la información pero sin generar análisis predictivo o estratégico sobre la misma.

Este recorrido secuencial, desde herramientas globales hasta la realidad local, evidencia el problema concreto que aborda este Trabajo de Grado: en Colombia existe un alto volumen de datos estatales disponibles, pero faltan sistemas autónomos que traduzcan esa información en decisiones estratégicas para las fábricas de software. Mientras las soluciones locales se limitan a emitir notificaciones y las investigaciones internacionales aplican IA solo para verificar documentos o asistir al Estado, el Producto Mínimo Viable (MVP) propuesto llena este vacío tecnológico ofreciendo un análisis desde la perspectiva del proveedor. La integración de un agente de recuperación, un extractor de requisitos LLM+RAG y un evaluador de cumplimiento que contrasta dichos requisitos contra el portafolio interno de la empresa, todo orquestado bajo la norma ISO/IEC/IEEE 42010:2022 y prácticas DevSecOps, constituye una solución innovadora que resuelve de forma directa las limitaciones de los antecedentes analizados.

1.2 PLANTEAMIENTO DEL PROBLEMA

La contratación pública constituye uno de los principales mecanismos mediante los cuales el Estado adquiere bienes y servicios para el cumplimiento de sus funciones administrativas, sociales y económicas. En Colombia, este proceso ha alcanzado un alto nivel de digitalización a través de plataformas transaccionales como SECOP II y TVEC, consolidando un mercado superior a los $174,8 billones de pesos. No obstante, la magnitud de este volumen de contratación contrasta con las barreras operativas que enfrentan las empresas para identificar, analizar y aprovechar de manera efectiva dichas oportunidades de negocio.

El problema central radica en que el análisis de la información publicada en los sistemas estatales exige la revisión manual de grandes volúmenes de documentación técnica, contractual y presupuestal por cada convocatoria. Las causas principales de esta situación se deben a la carencia de mecanismos tecnológicos automatizados que logren integrar y cruzar los datos abiertos de contratación con los recursos internos de las organizaciones, tales como su portafolio de servicios, experiencia técnica y capacidades operativas. Esta brecha de adopción tecnológica se ve intensificada por un mercado empresarial dinámico y en constante expansión, donde la creación de miles de nuevas empresas incrementa la presión competitiva. Como consecuencia, las áreas comerciales de las organizaciones experimentan una alta inversión de tiempo y recursos especializados, lo que incrementa los costos operativos y limita la capacidad de respuesta frente a convocatorias relevantes. Asimismo, la dependencia exclusiva de la lectura humana propicia errores de apreciación y postulaciones a procesos licitatorios que no se ajustan a las capacidades reales de la empresa, derivando en intentos fallidos que elevan el riesgo económico y vulneran la competitividad organizacional.

¿De qué manera la implementación de un sistema multi-agente, ante la ineficiencia operativa generada por el análisis manual de pliegos de condiciones, permite optimizar la evaluación estratégica de oportunidades de negocio en el mercado de contratación pública de Colombia para las fábricas de software?

1.3 JUSTIFICACIÓN

Importancia, pertinencia y valor agregado El desarrollo de este Producto Mínimo Viable (MVP) es pertinente porque da respuesta a la incapacidad operativa actual de las fábricas de software para procesar manualmente el abrumador volumen de datos comerciales y oportunidades de negocio. La magnitud de este problema se evidencia económicamente en el mercado estatal: para el año 2025, el 89,7% de la contratación pública gestionada en Colombia a través de plataformas como SECOP II y TVEC alcanzó un valor superior a los $174,8 billones de pesos. Paralelamente, el contexto del sector privado refleja un mercado amplio y dinámico; según datos del DANE, las ventas reales del comercio minorista aumentaron un 11,0% en diciembre de 2025, y los reportes del RUES indican que en el primer semestre de ese mismo año se registraron 173.907 nuevas empresas, muchas de ellas en el sector servicios.
Ante esta expansión comercial, el proyecto prioriza la investigación de una solución tecnológica sustentada en la urgencia del mercado hacia la Inteligencia Artificial generativa, un sector que atrajo $33.900 millones de dólares en financiación en 2024 y proyecta alcanzar un valor de $109.370 millones para 2030. El valor agregado del estudio radica en dotar a las organizaciones de un sistema capaz de automatizar el análisis y cruce de estas oportunidades con sus capacidades internas. Esto reduce el tiempo que el equipo comercial dedica a analizar pliegos, disminuye los costos operativos y evita el alto riesgo de postularse a licitaciones inviables.

Contribución a los Objetivos de Desarrollo Sostenible (ODS)
El desarrollo de esta solución tecnológica se alinea con la Agenda 2030, aportando de forma directa al cumplimiento de los siguientes Objetivos de Desarrollo Sostenible:

- ODS 8 (Trabajo decente y crecimiento económico): El proyecto impacta directamente la Meta 8.2, cuyo propósito es "lograr niveles más elevados de productividad económica mediante la diversificación, la modernización tecnológica y la innovación". La automatización del análisis documental libera horas de trabajo humano repetitivo y agobiante, elevando significativamente la productividad de los equipos comerciales y analíticos.

- ODS 9 (Industria, innovación e infraestructura): El sistema contribuye a la Meta 9.4, enfocada en la modernización tecnológica para hacer a las industrias más sostenibles, promoviendo la "adopción de tecnologías y procesos industriales limpios y ambientalmente racionales". El uso de Inteligencia Artificial evita la impresión física de miles de páginas de pliegos gubernamentales, a la vez que fomenta la innovación digital en el ciclo de compras empresariales.

Articulación con Políticas Públicas y el CONPES
La pertinencia del proyecto se consolida al articularse con los lineamientos del CONPES 3975 de 2019 ("Política Nacional para la Transformación Digital e Inteligencia Artificial"). Este documento estratégico prioriza el uso de datos abiertos gubernamentales y la adopción de tecnologías de la Cuarta Revolución Industrial para generar valor social y económico. El ecosistema multi-agente propuesto materializa los objetivos de este documento al aprovechar la infraestructura de datos abiertos de SECOP II, construyendo sobre ellos una solución de IA

que dinamiza la participación transparente y competitiva del sector privado en las compras del Estado.
Aporte a la Línea de Investigación Institucional Este trabajo de grado se inscribe y contribuye a las líneas de investigación del programa de Ingeniería de Sistemas y Computación de la Universidad Católica de Colombia, articulándose con los ejes temáticos de *Ingeniería de Software* e *Inteligencia Artificial*. El proyecto fortalece la actividad investigativa de la facultad al demostrar cómo integrar modelos de lenguaje (LLMs) y arquitecturas multi-agente en entornos comerciales e industriales reales.
Justificación y valor agregado metodológico Frente al riesgo implícito que conlleva la adopción de IA, este proyecto fundamenta su viabilidad técnica en la estricta aplicación de metodologías y estándares internacionales, lo que otorga un alto valor agregado a la disciplina:

- Ingeniería de Requisitos: Aplicación de la norma ISO/IEC/IEEE 29148:2018 para levantar especificaciones sin ambigüedad.
- Arquitectura: Diseño bajo la norma ISO/IEC/IEEE 42010:2022 y el Modelo C4 (Contexto, Contenedores, Componentes, Código), permitiendo escalar el sistema ordenadamente.
- Desarrollo Seguro: Adopción del marco Scrum con prácticas DevSecOps (Shift-Left) desde las primeras etapas de codificación para prevenir vulnerabilidades.
- Validación: Ejecución de pruebas QA/QC con base en la norma NTC-ISO/IEC 25010 para garantizar el rendimiento y la adecuación funcional del MVP.

1.4 OBJETIVOS

1.4.1 Objetivo General.

Desarrollar un Producto Mínimo Viable (MVP) para la identificación de requisitos de licitaciones públicas y la evaluación de su cumplimiento por parte de las capacidades de una fábrica de software mediante un sistema de inteligencia artificial multi-agente para optimizar decisiones comerciales Y reducir costos operativos.

1.4.2 Objetivos Específicos.

1. Definir los RF y RNF del sistema multi-agente, aplicando los atributos de calidad de las secciones 5.2, 5.4 y Apéndice A de la norma ISO/IEC/IEEE 29148:2018, para establecer las bases técnicas y reglas de negocio.

2. Diseñar la arquitectura de software del sistema multi-agente basándose en el estándar ISO/IEC/IEEE 42010:2022 con la identificación de stakeholders, concerns y viewpoints aplicando los Niveles 1, 2 y 3 del modelo C4, para estructurar una solución escalable en el desarrollo del MVP.

3. Implementar el Producto Mínimo Viable (MVP) del sistema multi-agente, implementando marco de trabajo ágil y prácticas DevSecOps, para materializar los algoritmos de análisis estratégico, la integración de fuentes de datos y la interfaz de usuario requerida.

4. Evaluar el MVP del sistema multi-agente a partir de un plan de pruebas de aseguramiento de calidad y control de calidad, para la verificación de su desempeño.

1.5 MARCOS DE REFERENCIA

1.5.1 Marco Conceptual.

El desarrollo de este trabajo de investigación no se concibe a partir de conceptos tecnológicos aislados, sino como un ecosistema interconectado donde la necesidad del negocio dicta la estructura del software, y los estándares de ingeniería garantizan su viabilidad.

Perspectivas académicas e interpretación de los conceptos principales

Para la consecución de los resultados esperados, los conceptos que componen este mapa relacional se interpretan bajo las siguientes perspectivas aplicadas a la ingeniería de software:

1. Inteligencia Artificial Multi-Agente (Perspectiva de Soporte a Decisiones): En el contexto de este proyecto, la IA no se concibe como un simple modelo de procesamiento de lenguaje natural como chatbot, sino bajo la perspectiva de la "IA Agéntica". Se interpreta como un sistema de entidades autónomas por agentes con roles segregados que colaboran para resolver un problema complejo de clasificación de información y *scoring* de viabilidad comercial, lo cual es el fin último del MVP.
2. Arquitectura Basada en Estándares: Los conceptos de Requisitos Funcionales (RF), Requisitos No Funcionales (RNF) y el Modelo C4 no se abordan como mera documentación burocrática, sino como el marco estructural indispensable que dicta el comportamiento de los agentes. Se asume la perspectiva académica de que el diseño

arquitectónico (Objetivos 1 y 2) previene la deuda técnica y garantiza que el sistema pueda escalar en el futuro sin necesidad de ser rediseñado.

3. DevSecOps y Shift-Left (Perspectiva de Construcción Segura): El desarrollo del sistema se concibe bajo una perspectiva preventiva. DevSecOps no se interpreta únicamente como la automatización de despliegues, sino como una cultura operativa donde las pruebas de seguridad estática y análisis de vulnerabilidades se "desplazan a la izquierda" (Shift-Left), integrándose desde las fases más tempranas de la codificación en los Sprints ágiles.
4. Aseguramiento y Control de Calidad (Perspectiva de Validación Continua): Finalmente, los conceptos de QA (enfoque preventivo) y QC (enfoque correctivo) se aplican desde la óptica del estándar NTC-ISO/IEC 25010. La calidad del sistema multi-agente se interpretará midiendo específicamente su "Adecuación Funcional" donde exactitud en la evaluación de los pliegos frente a las capacidades de la empresa y su "Eficiencia de Desempeño" en uso óptimo de recursos en la nube y tiempos de respuesta.

1.5.2 Marco Teórico.

El presente trabajo de investigación se fundamenta en un conjunto de teorías y estándares de la ingeniería de software y la inteligencia artificial, extraídos de literatura arbitrada reciente. Estas bases teóricas no solo delimitan el alcance del proyecto, sino que justifican técnica y metodológicamente la ejecución de cada uno de los objetivos específicos planteados.

Teoría de Sistemas Multi-Agente e Inteligencia Artificial Generativa
La teoría de los Sistemas Multi-Agente postula que la resolución de problemas lógicos complejos puede optimizarse distribuyendo tareas cognitivas entre múltiples agentes autónomos, cada uno dotado o asignado de un rol especializado dentro de un flujo de trabajo [18]. En el contexto de la contratación pública, autores como Waseem et al. [32] y Sofat & Sodhi [29] teorizan que la evaluación estratégica de licitaciones (*bid evaluation*) y el procesamiento de requerimientos técnicos pueden automatizarse exitosamente mediante modelos de lenguaje (LLMs). Esta teoría fundamenta la viabilidad de nuestro núcleo tecnológico, justificando la creación de un sistema donde entidades de software extraigan y evalúen pliegos de condiciones sin depender de la intervención humana constante.

Teoría de la Ingeniería de Requisitos y Arquitectura de Software
La concepción del software de misión crítica exige que los requerimientos no sean meras expectativas del cliente, sino especificaciones formales, verificables y trazables, tal como lo establece el estándar ISO/IEC/IEEE 29148:2018. Teóricos como Ferrari et al. [12] sostienen que una rigurosa elicitación de requisitos es el pilar fundamental para evitar fallos sistémicos tempranos. Sobre esta base, el diseño del sistema se fundamenta en la teoría de la arquitectura de software de Bass et al. [7], la cual determina que la estructura subyacente de un sistema es la que garantiza sus atributos de calidad. Para materializar estos conceptos, el proyecto justifica la adopción de la norma ISO/IEC/IEEE 42010:2022 y los niveles de abstracción jerárquica del Modelo C4 (Contexto, Contenedores, Componentes y Código),

asegurando que el ecosistema agéntico sea comprensible y escalable [20].

Teoría del Desarrollo Seguro Continuo y DevSecOps
Para la construcción técnica del Producto Mínimo Viable, el proyecto adopta los principios de iteración del marco ágil Scrum, complementados estrechamente con la teoría de operaciones de seguridad de software, conocida como DevSecOps. Autores como Amaro et al. [4] y Ziegler et al. [36] fundamentan académicamente el enfoque metodológico de "desplazamiento a la izquierda" (*Shift-Left*). Esta teoría postula que la integración de pruebas estáticas y escaneos de vulnerabilidades desde la primera línea de código es estadísticamente más eficiente y menos costosa que la mitigación reactiva en etapas de producción. Este paradigma teórico justifica la configuración de un entorno de integración continua para el desarrollo del software.

Teoría de Validación y Aseguramiento de Calidad de Software
Evaluar un sistema basado en Inteligencia Artificial exige criterios distintos a los del testing determinista. Amalfitano et al. [3] y Butt et al. [10] muestran que los sistemas de Machine Learning y las arquitecturas LLM necesitan controles que midan el grado de certidumbre de las respuestas y el consumo de recursos computacionales. La validación de este proyecto se apoya en la familia SQuaRE (ISO/IEC 25000), cuyo marco de calidad cambió hace poco: la norma ISO/IEC 25010:2011 fue cancelada y su contenido pasó a tres documentos. La norma ISO/IEC 25010:2023 conserva el modelo de calidad de producto y lo lleva a nueve características, con la incorporación de Safety y el reemplazo de Usabilidad y Portabilidad por Capacidad de Interacción y Flexibilidad. La norma ISO/IEC 25019:2023 recoge el modelo de calidad en uso, ahora con tres características: beneficialidad, libertad de riesgo y aceptabilidad. La norma ISO/IEC 25002:2024 define qué es un modelo de calidad y cómo se usa. Como el objeto de evaluación es el prototipo en sí y no la experiencia del usuario final en operación, la validación toma el modelo de calidad de producto de la norma ISO/IEC 25010:2023 y las pautas de aplicación de la norma ISO/IEC 25002:2024. Sobre esa base se ejecuta un plan de Aseguramiento (QA) y Control de Calidad (QC) limitado a dos características que siguen vigentes en la edición 2023. La primera es la Adecuación Funcional, verificada por su subcaracterística de corrección funcional, esto es, la exactitud de las evaluaciones de los pliegos. La segunda es la Eficiencia de Desempeño, verificada por el comportamiento temporal y la utilización de recursos del prototipo.

1.5.3 Marco Jurídico.

1.6 DISEÑO METODOLÓGICO

Para el desarrollo del presente proyecto, se adopta un enfoque de investigación aplicada, estructurado bajo un ciclo de vida de desarrollo de software ágil basado en el marco de trabajo Scrum. Esta elección metodológica permite una entrega iterativa e incremental del Producto Mínimo Viable (MVP), facilitando la adaptación continua a los requisitos y estándares de calidad definidos. El desarrollo se organiza en cuatro fases principales, cada una alineada a los objetivos específicos del proyecto:

Fase 1: Elicitación y Análisis de Requisitos
En esta fase se establecen los cimientos funcionales y no funcionales del sistema. Se aplica la norma ISO/IEC/IEEE 29148:2018 para garantizar que los requerimientos sean claros, verificables y trazables.

- Entregables: Documento de Especificación de Requisitos de Software con la matriz de RF y RNF.
- Herramientas: Matriz de gestión de requisitos, herramientas de modelado de Historias de Usuario con criterios Gherkin.

Fase 2: Diseño de la Arquitectura de Software
Se procede a la estructuración lógica del ecosistema multi-agente. Se utiliza el estándar ISO/IEC/IEEE 42010:2022 junto con la metodología del Modelo C4 (Contexto, Contenedores, Componentes, Código) para asegurar la escalabilidad y coherencia técnica.

- Entregables: Documento de diseño arquitectónico y diagramas jerárquicos C4.
- Herramientas: Software de modelado arquitectónico C4 Model

Fase 3: Implementación y Desarrollo del MVP
Esta fase materializa el diseño anterior mediante iteraciones bajo prácticas DevSecOps. Se integra la seguridad desde el código para mitigar vulnerabilidades desde las etapas tempranas de codificación.

- Entregables: Prototipo funcional (MVP) desplegado, integrado con las fuentes de datos y la interfaz de usuario.
- Herramientas: Entornos de desarrollo integrados, sistemas de control de versiones con AzureDevOps y pipelines de integración continua para automatización de pruebas de seguridad estática.

Fase 4: Evaluación de Calidad
Se realiza la validación técnica del MVP mediante un plan de pruebas estructurado bajo la norma NTC-ISO/IEC 25010:2023. El enfoque se centra en la adecuación funcional y la eficiencia de desempeño.

- Entregables: Informe de aseguramiento QA y control de calidad QC.
- Herramientas: Suites de pruebas automatizadas y herramientas de monitoreo de telemetría correlacionada por tarea por definir.

2. DEFINICIÓN DE REQUISITOS DEL SISTEMA MULTI-AGENTE

Este capítulo desarrolla el primer objetivo específico del proyecto: definir los requisitos funcionales y no funcionales del sistema multi-agente aplicando los atributos de calidad de las secciones 5.2 y 5.4 y del Apéndice A de la norma ISO/IEC/IEEE 29148:2018. El resultado se presenta de lo general a lo particular: primero las épicas que delimitan la cadena de valor del MVP, luego las historias de usuario que las concretan y, por último, los requisitos funcionales y no funcionales arquitectónicamente significativos, es decir, aquellos que condicionan la estructura del sistema. En este capítulo solo se detallan esos requisitos significativos; la especificación completa, que incluye el resto de requisitos funcionales, las fichas individuales de cada requisito y los escenarios de aceptación, se entrega en el Anexo A.

El propósito de la fase no fue redactar una lista de funciones deseables, sino construir una línea base verificable, es decir un conjunto de enunciados sobre los cuales sea posible decidir si el sistema cumple o no cumple. Esa exigencia pesa de manera particular en un sistema que delega el análisis documental a modelos de lenguaje: un enunciado como "el sistema debe analizar correctamente el pliego" no admite prueba, y sin prueba no hay forma de distinguir un agente que funciona de uno que produce respuestas plausibles pero equivocadas. La línea base resultante se resume en la Tabla 1.

Tabla 1. Resumen de la línea base de requisitos

Artefacto	Cantidad	Contenido
Épicas	5	Agrupan las historias de usuario por etapa de la cadena de valor del MVP: portafolio de capacidades, vigilancia de oportunidades, extracción y revisión de requisitos, evaluación y recomendación, y entrega de resultados.
Historias de usuario (HU)	10	Redactadas en formato Como–Quiero–Para, con trazabilidad a RF y RNF, prioridad MoSCoW, estimación de 57 puntos de historia en total y verificación INVEST.
Escenarios de aceptación	38	Entre tres y cuatro por historia, en notación Gherkin: 10 de camino principal, 17 alternos y 11 de excepción.
Requisitos funcionales (RF)	43	Siete bloques temáticos: acceso y sesión, portafolio de capacidades, monitoreo de oportunidades, procesamiento y extracción documental, evaluación y puntaje, reporte y gestión comercial, auditoría y observabilidad. Se clasifican en 31 Debe, 10 Debería y 2 Podría. Nueve son arquitectónicamente significativos (siete confirmados y dos pendientes de evidencia).
Requisitos no funcionales (RNF)	10	Cubren las subcaracterísticas comprometidas del modelo de calidad ISO/IEC 25010:2023: comportamiento temporal, capacidad, interoperabilidad, disponibilidad, confidencialidad, integridad, autenticidad, tolerancia a fallos y escalabilidad, cada uno con su métrica, su umbral y su método de verificación. Los diez son arquitectónicamente significativos.

Fuente: elaboración propia.

2.1 MÉTODO Y CRITERIOS DE ESPECIFICACIÓN

La elicitación combinó tres fuentes: la estructura documental de los procesos publicados en SECOP II, que determina qué información puede extraerse y con qué grado de estructura; el proceso comercial de la fábrica de software, del que provienen los roles de Analista Comercial y Gerente Comercial y la separación entre la recomendación automática y la decisión humana; y los antecedentes revisados en el estado del arte. Cada requisito registra su fuente y su justificación, lo que constituye la evidencia del atributo de necesidad que exige la norma: un requisito cuya eliminación ninguna parte interesada reclama no debería ocupar un lugar en la línea base.

La versión inicial contenía 30 requisitos funcionales y la revisión bajo el atributo de singularidad los elevó a 43 sin agregar alcance nuevo. La norma exige que cada requisito enuncie una sola capacidad, porque un enunciado compuesto no puede aprobarse ni rechazarse de forma unívoca: si el sistema cumple la mitad de lo que declara, el resultado de la verificación queda indeterminado. Así, el requisito que reunía habilitar usuarios, deshabilitarlos y asignarles permisos se separó en RF-002, RF-003 y RF-004, y el que reunía la configuración del puntaje y su versionado se separó en RF-032 y RF-033, porque un sistema puede permitir configurar sin conservar el histórico y esa diferencia decide si los análisis ya emitidos siguen siendo reproducibles. La división expuso además una omisión de fondo: ningún requisito especificaba la creación de usuarios pese a que la asignación de permisos y la habilitación la presuponían, vacío que se cubrió con RF-001.

En sentido inverso, los requisitos no funcionales pasaron de treinta enunciados a ocho. La mayoría de los originales fijaba coberturas de prueba, tiempos de despliegue o condiciones de integración de código, es decir describía el proceso de desarrollo y no una propiedad del producto en operación; esos enunciados no se descartaron, sino que se reubicaron como criterios de aceptación de los requisitos funcionales correspondientes o como métricas del plan de calidad del cuarto objetivo específico, que es donde admiten verificación. Los restantes separan la descripción del atributo de su cuantificación: la propiedad de calidad se enuncia en lenguaje de negocio, mientras que la métrica, la unidad, el umbral, la condición de medición y el método ocupan campos independientes. Por esa razón RNF-005 exige confidencialidad sobre los datos del portafolio sin nombrar el algoritmo; el cifrado AES-256 en reposo y TLS 1.3 en tránsito es la decisión arquitectónica derivada, y puede cambiar sin que cambie el requisito. Una revisión posterior sumó dos requisitos de autenticidad, RNF-009 y RNF-010, que exigen resolver la autorización del lado del servidor y retirar toda capacidad de operación a una sesión cerrada, expirada o perteneciente a un usuario deshabilitado. Se incorporaron porque los requisitos funcionales de acceso y de sesión no estaban comprometidos por ningún atributo de calidad medible, de modo que su cumplimiento no admitía verificación. Con ellos el catálogo queda en diez.

La especificación se registra en un libro de trabajo con catálogos controlados, columnas derivadas y validación automática de las referencias cruzadas antes de guardar cada versión. Esa mecanización permite que la comprobación exigida por el primer indicador clave de desempeño se apoye en conteos reproducibles y no en una revisión manual. Ese libro de trabajo constituye el Anexo A del presente documento.

2.2 ÉPICAS DEL SISTEMA

Las épicas se definieron a partir de la secuencia que sigue la fábrica de software frente a una licitación pública: disponer de un portafolio vigente contra el cual comparar, detectar las oportunidades pertinentes, convertir el pliego en una lista de requisitos confiable, evaluar el cumplimiento y entregar un resultado que sustente la decisión comercial. Cada épica agrupa las historias de usuario que materializan una de esas etapas, de modo que el catálogo recorre el proceso completo sin solapar responsabilidades entre épicas. La Tabla 2 presenta el catálogo con el tamaño estimado de cada épica y el sprint en que se planifican sus historias.

Tabla 2. Catálogo de épicas del sistema multi-agente

ID	Épica	Alcance	Historias	Puntos	Sprint
EP-01	Portafolio de capacidades	Registro y versionado de las tecnologías, la experiencia y los casos de éxito de la fábrica, que conforman la base de conocimiento contra la cual se evalúa cada licitación.	HU-001	5	1
EP-02	Vigilancia de oportunidades	Recuperación del expediente documental de los procesos de SECOP II y notificación de las licitaciones que coinciden con los perfiles de búsqueda.	HU-002, HU-004	13	1 y 2
EP-03	Extracción y revisión de requisitos	Conversión del pliego en una lista estructurada de requisitos y su aprobación por el analista antes de la evaluación.	HU-005, HU-006	13	2
EP-04	Evaluación y recomendación	Dictamen de cumplimiento por requisito, puntaje global de 0 a 100 y recomendación explicada de ofertar, no ofertar o solicitar revisión.	HU-003, HU-007, HU-008	16	3
EP-05	Entrega de resultados	Informe en PDF verificable y tablero consolidado para el seguimiento del embudo comercial.	HU-009, HU-010	10	4
Total	**5 épicas**		**10 historias**	**57**	**4 sprints**

Fuente: elaboración propia.

La mayor carga de estimación se concentra en EP-04, con 16 puntos, porque reúne la lógica que el sistema delega a los agentes y la que traduce su salida en una recomendación comercial. EP-01 es la de menor tamaño, pero antecede a todas las demás: sin un portafolio registrado no existe una referencia contra la cual evaluar ningún pliego.

2.3 HISTORIAS DE USUARIO Y CRITERIOS DE ACEPTACIÓN

La línea base se tradujo a diez historias de usuario agrupadas en cinco épicas, que recorren la cadena de valor del prototipo desde el registro del portafolio hasta el tablero consolidado. Cada historia declara los requisitos funcionales y no funcionales que la sustentan, su prioridad, su estimación en puntos y su verificación frente a los criterios INVEST; la trazabilidad hacia los requisitos no funcionales se validó de forma cruzada contra los requisitos funcionales que cada uno afecta, de manera que una historia no pueda declarar una restricción que no le

corresponde. La Tabla 3 presenta las historias agrupadas por épica.

Tabla 3. Historias de usuario agrupadas por épica

ID	Historia de usuario	RF	RNF	Prioridad	Puntos	Escenarios
EP-01 Portafolio de capacidades						
HU-001	Como Analista Comercial, quiero registrar y actualizar las tecnologías que maneja la fábrica, los años de experiencia y los casos de éxito, para que el sistema disponga de una base de conocimiento vigente contra la cual evaluar las licitaciones.	RF-008, RF-009	RNF-005	Debe	5	4
EP-02 Vigilancia de oportunidades						
HU-002	Como Analista Comercial, quiero registrar el enlace de un proceso de SECOP II y que el sistema recupere sus pliegos y anexos, para disponer del expediente completo sin descargar cada documento a mano.	RF-011, RF-021	RNF-003	Debe	8	4
HU-004	Como Analista Comercial, quiero que el sistema me notifique cuando una licitación publicada en SECOP II coincida con los criterios de mi perfil de búsqueda, para no revisar la plataforma manualmente todos los días.	RF-012, RF-015, RF-016, RF-017	RNF-003	Debería	5	4
EP-03 Extracción y revisión de requisitos						
HU-005	Como Analista Comercial, quiero que el agente extractor procese el pliego de la licitación seleccionada y entregue una lista estructurada de requisitos técnicos, financieros, jurídicos, de experiencia y de personal, para evitar la lectura manual de documentos extensos.	RF-023, RF-029	RNF-001	Debe	8	4
HU-006	Como Analista Comercial, quiero revisar, corregir, agregar o eliminar los requisitos extraídos antes de que se evalúen, para que la evaluación parta de una lista confiable y no de una salida automática sin verificar.	RF-030	N/A	Debe	5	4
EP-04 Evaluación y recomendación						
HU-003	Como Gerente Comercial, quiero recibir una recomendación de ofertar, no ofertar o solicitar revisión con la justificación que la sustenta, para decidir con rapidez si la empresa invierte recursos en armar la propuesta o descarta la licitación.	RF-035	RNF-001, RNF-006	Debe	5	4
HU-007	Como Analista Comercial, quiero que el sistema emita para cada requisito aprobado un dictamen de cumplimiento con su evidencia y su justificación, para saber en qué exigencias concretas del pliego la fábrica queda corta.	RF-031	RNF-001	Debe	8	3
HU-008	Como Gerente Comercial, quiero ver un puntaje de cumplimiento entre 0 y 100 con el aporte de cada categoría de requisitos, para comparar unas oportunidades con otras bajo un mismo criterio.	RF-034	RNF-006	Debe	3	4
EP-05 Entrega de resultados						

ID	Historia de usuario	RF	RNF	Prioridad	Puntos	Escenarios
HU-009	Como Gerente Comercial, quiero descargar un informe en PDF con los dictámenes, el puntaje, la recomendación y las versiones utilizadas, para sustentar la decisión ante el comité comercial con un documento verificable.	RF-037	RNF-006	Debe	5	3
HU-010	Como Gerente Comercial, quiero consultar en un tablero filtrable las oportunidades analizadas con su estado, su puntaje, su recomendación y su decisión, para seguir el estado del embudo comercial sin pedir reportes uno por uno.	RF-038	RNF-004	Debería	5	4

Fuente: elaboración propia.

Los 38 escenarios de aceptación se redactaron en notación Gherkin por dos razones. La primera es de verificabilidad: al declarar el resultado en términos observables, cada escenario se convierte en un caso de prueba durante el cuarto objetivo específico sin una traducción intermedia que introduzca interpretación. La segunda es de consistencia: los escenarios se derivaron de los flujos principales y de las reglas de negocio ya establecidos en la matriz de requisitos, de modo que por construcción no pueden contradecir al requisito que verifican. La cobertura se orientó de forma deliberada hacia los caminos alternos y de excepción, porque en un sistema que analiza documentos que pueden estar incompletos, desactualizados o ilegibles, el comportamiento ante la información faltante es el que decide si la salida puede sustentar una decisión comercial. Por esa razón, de los 38 escenarios solo diez describen el camino principal, mientras que diecisiete cubren caminos alternos y once cubren excepciones; su redacción completa se encuentra en el Anexo A.

2.4 REQUISITOS FUNCIONALES ARQUITECTÓNICAMENTE SIGNIFICATIVOS

No todos los requisitos pesan igual sobre la arquitectura. Un enunciado se clasificó como arquitectónicamente significativo cuando obliga a particionar el sistema en unidades de despliegue distintas, atraviesa de forma transversal varios componentes, restringe la selección de tecnología o su reversión tardía implica rediseñar en lugar de modificar. En los requisitos funcionales el criterio se aplicó de forma estricta: es significativo el requisito que constituye por sí mismo la base de un escenario arquitectónico, y no aquel que solo aparece en la trazabilidad de un atributo de calidad, porque en ese caso el peso estructural lo aporta el requisito no funcional correspondiente. Bajo ese criterio, 9 de los 43 requisitos funcionales resultan significativos, como muestra la Tabla 4. La columna ASR indica el escenario con el que el segundo objetivo específico formaliza cada requisito.

Tabla 4. Requisitos funcionales arquitectónicamente significativos

ID	Requisito	Prioridad	HU	Implicación arquitectónica	ASR
RF-024	Neutralización de instrucciones maliciosas incrustadas en los documentos	Debe	—	Obliga a separar el canal de ingesta documental del canal de instrucciones del agente, porque un pliego puede contener texto dirigido al modelo y no al lector.	ASR-10

ID	Requisito	Prioridad	HU	Implicación arquitectónica	ASR
RF-025	Ejecución del análisis en segundo plano	Debe	—	Impone una partición entre la interfaz síncrona que recibe la solicitud y el motor de procesamiento que la ejecuta, con una cola de trabajo entre ambos.	ASR-11
RF-026	Consulta del estado de la tarea de análisis	Debe	—	Exige un identificador de tarea persistente y consultable, independiente del proceso que ejecuta el análisis.	ASR-11
RF-028	Control del consumo de tokens del agente de inteligencia artificial	Debe	—	Requiere un punto único de control por el que pase toda invocación al modelo de lenguaje antes de ejecutarse.	ASR-13
RF-029	Extracción estructurada de los requisitos del pliego	Debe	HU-005	Define el contrato de datos de salida del agente extractor, que es a la vez la entrada de la revisión humana y del agente evaluador.	ASR-12
RF-030	Revisión humana de los requisitos extraídos	Debe	HU-006	Obliga a que el flujo multi-agente se detenga y conserve su estado mientras espera la aprobación del analista.	ASR-12
RF-031	Emisión del dictamen de cumplimiento por requisito	Debe	HU-007	Restringe al agente evaluador a operar solo sobre la lista aprobada, lo que exige un orquestador con estado compartido de la tarea.	ASR-12
RF-032	Configuración de las ponderaciones y umbrales del puntaje	Debe	—	Separa las reglas del puntaje del cálculo, en un mecanismo configurable por el responsable autorizado.	ASR-14 (por validar)
RF-033	Versionado de las reglas del puntaje	Debe	—	Obliga a conservar la versión de las reglas empleada en cada análisis para que los resultados sigan siendo reproducibles.	ASR-14 (por validar)

Fuente: elaboración propia.

Cuatro de estos requisitos, RF-024, RF-025, RF-026 y RF-028, no se asocian a una historia de usuario porque describen capacidades transversales de la plataforma que ningún rol de negocio solicita de forma directa; su verificación se apoya en el escenario ASR correspondiente y en el plan de pruebas del cuarto objetivo específico. RF-032 y RF-033 permanecen como pendientes, ya que su peso arquitectónico depende de la matriz de ponderaciones y umbrales que el cliente interno aún no aprueba. En sentido contrario, RF-041, exportación de oportunidades a CSV, se evaluó como candidato y se descartó, porque se resuelve con una librería de serialización dentro de un componente y no afecta la estructura del sistema. Los 34 requisitos funcionales restantes no condicionan la arquitectura y se documentan con su especificación completa en el Anexo A.

2.5 REQUISITOS NO FUNCIONALES ARQUITECTÓNICAMENTE SIGNIFICATIVOS

La depuración descrita en el apartado 2.1 retiró del catálogo los enunciados que describían el proceso de desarrollo y conservó únicamente atributos de calidad del producto en operación. Como consecuencia, los diez requisitos no funcionales condicionan la arquitectura y cada uno origina un escenario ASR. La Tabla 5 los presenta con su atributo de calidad, su umbral de aceptación y los requisitos funcionales que restringen; la métrica, la condición de medición y el método de verificación de cada uno se encuentran en el Anexo A.

Tabla 5. Requisitos no funcionales arquitectónicamente significativos

ID	Requisito	Atributo ISO/IEC 25010:2023	Umbral de aceptación	RF afectados	ASR
RNF-001	Tiempo de respuesta en la evaluación de pliegos	Eficiencia de desempeño / Comportamiento temporal	≤ 10 minutos por licitación en el percentil 90	RF-025, RF-029, RF-031, RF-034, RF-035, RF-037	ASR-01
RNF-002	Soporte de evaluaciones concurrentes	Eficiencia de desempeño / Capacidad	10 evaluaciones simultáneas en el percentil 95	RF-011, RF-025, RF-026, RF-031	ASR-02
RNF-003	Interoperabilidad con fuentes de datos estatales	Compatibilidad / Interoperabilidad	≥ 98 % de lectura e interpretación exitosa del esquema	RF-011, RF-012, RF-021	ASR-03
RNF-004	Recuperación del sistema ante caídas críticas	Fiabilidad / Disponibilidad	Recuperación ≤ 4 horas; pérdida máxima de datos ≤ 12 horas	RF-025, RF-026, RF-027, RF-038	ASR-04
RNF-005	Protección de datos estratégicos empresariales	Seguridad / Confidencialidad	100 % de la información sensible protegida	RF-002, RF-005, RF-008, RF-009, RF-010	ASR-05
RNF-006	Inmutabilidad de los reportes de viabilidad generados	Seguridad / Integridad	100 % de registros comprobables frente a alteraciones	RF-034, RF-035, RF-037, RF-042	ASR-06
RNF-007	Tolerancia a la caída de servicios y API externas	Fiabilidad / Tolerancia a fallos	0 % de interrupciones del núcleo del sistema	RF-025, RF-027, RF-029, RF-031	ASR-07
RNF-008	Adaptación a aumentos de carga licitatoria	Flexibilidad / Escalabilidad	0 archivos de código modificados para duplicar la capacidad	RF-025, RF-026, RF-028	ASR-08
RNF-009	Rechazo de accesos no autorizados	Seguridad / Autenticidad	100 % de intentos no autorizados rechazados	RF-002, RF-004, RF-005, RF-032, RF-038, RF-039	ASR-09
RNF-010	Vigencia de las sesiones de usuario	Seguridad / Autenticidad	0 sesiones sin vigencia con capacidad de operación	RF-004, RF-005, RF-006, RF-007	ASR-09

Fuente: elaboración propia.

Dos casos ilustran el alcance de ese peso. El percentil 90 de diez minutos por pliego fijado en RNF-001 condiciona la topología de ejecución, porque obliga a paralelizar el procesamiento de fragmentos y descarta una arquitectura de proceso único que recorra el documento de forma secuencial. RNF-005, por su parte, alcanza la capa de transporte, la persistencia del índice vectorial, la autenticación y el contenido que se envía al proveedor del modelo de lenguaje, de modo que su impacto es transversal a toda la estructura. La prioridad relativa entre estos atributos no se resuelve en este capítulo: corresponde al árbol de utilidad del segundo objetivo específico.

2.6 RESULTADO DEL OBJETIVO Y ARTICULACIÓN CON EL SEGUNDO OBJETIVO

La línea base declara de forma explícita los parámetros que el cliente interno aprobado, entre ellos las ponderaciones y umbrales del puntaje de cumplimiento y la matriz de roles y permisos, en lugar de fijarlos por conveniencia. La norma admite el pendiente siempre que quede

registrado y asociado al requisito que afecta; lo que no admite es un requisito que aparente estar cerrado sobre un parámetro que nadie ha decidido, porque esa apariencia se traslada al diseño y allí se convierte en una suposición del desarrollador. Varios de estos pendientes, en particular la matriz de roles, deben resolverse antes del cierre del segundo objetivo específico.

El resultado de este objetivo no es un fin en sí mismo, sino la entrada del segundo objetivo específico. Los requisitos significativos identificados en los apartados 2.4 y 2.5 se formalizan allí como escenarios de calidad de seis partes, a saber, fuente, estímulo, entorno, artefacto, respuesta y medida de la respuesta, y reciben un dictamen sobre su impacto en la estructura del sistema. Según su naturaleza siguen tres rutas, los requisitos no funcionales que admiten una comparación de prioridad alimentan el árbol de utilidad (Utility Tree), donde se ubican según su valor de negocio y su impacto arquitectónico los cuales expresan una obligación no negociable se tratan como restricciones transversales y los requisitos funcionales significativos se tratan como funciones críticas que anclan el diseño del sistema. La Tabla 6 muestra esa correspondencia.

Tabla 6. Correspondencia entre la línea base de requisitos y los insumos del segundo objetivo

Destino en el segundo objetivo	Requisitos de origen	Escenarios ASR	Cantidad
Árbol de utilidad (escenarios de calidad)	RNF-001, RNF-002, RNF-003, RNF-004, RNF-006, RNF-007, RNF-008, RNF-009, RNF-010	ASR-01 a ASR-04 y ASR-06 a ASR-09	8
Restricciones transversales	RNF-005, RF-028	ASR-05, ASR-13	2
Funciones críticas	RF-024, RF-025, RF-026, RF-029, RF-030, RF-031	ASR-10, ASR-11, ASR-12	3
Pendiente de evidencia	RF-032, RF-033	ASR-14	1
Descartado por no tener impacto estructural	RF-041	ASR-15	1
Total			**15**

Fuente: elaboración propia.

Con esa salvedad, el primer objetivo específico queda cumplido: cinco épicas, 10 historias de usuario, 38 escenarios de aceptación, 43 requisitos funcionales y 10 requisitos no funcionales expresados como atributos de calidad medibles, trazables entre sí y verificados contra los nueve atributos del Apéndice A de la norma ISO/IEC/IEEE 29148:2018, con su especificación completa en el Anexo A. Más allá del conteo, el resultado relevante es que el árbol de utilidad que se construye en el capítulo siguiente no parte de una intuición: cada escenario que prioriza responde a un requisito identificable de esta línea base, y cada requisito admite una prueba que decide si el sistema lo cumple.

3. DISEÑO DE LA ARQUITECTURA DE SOFTWARE DEL SISTEMA MULTI-AGENTE

Este capítulo desarrolla el segundo objetivo específico del proyecto: diseñar la arquitectura de software del sistema multi-agente con base en la norma ISO/IEC/IEEE 42010:2022, identificando partes interesadas, preocupaciones y puntos de vista, y representándola en los niveles 1, 2 y 3 del Modelo C4. El punto de partida es el árbol de utilidad que se construye con los requisitos significativos del capítulo anterior (Tabla 6). Sobre ese árbol se decide qué estilo arquitectónico adopta el MVP, cómo se coordinan sus agentes y qué tecnologías lo soportan. Cada decisión se registra junto con las alternativas que se evaluaron y la razón por la que se descartaron.

El capítulo sigue el orden en que se tomaron las decisiones. Primero se fija el marco de descripción que propone la norma (3.1) y se identifican las partes interesadas y sus preocupaciones (3.2). Luego se consolidan los impulsores arquitectónicos (3.3), se selecciona el estilo (3.4) y el patrón de coordinación de los agentes (3.5), y se registran las decisiones con sus contrapartidas (3.6) y las consideraciones que no aplican al MVP (3.7). Con las decisiones cerradas se definen los puntos de vista (3.8), se presentan las vistas modeladas en Structurizr (3.9 a 3.13) y se documentan las correspondencias que muestran la cobertura de cada escenario (3.14). El diseño de la interfaz de usuario, con sus maquetas, se desarrolla en apartados posteriores de este mismo objetivo a partir de lo que aquí se decide.

3.1 MARCO DE DESCRIPCIÓN ARQUITECTÓNICA SEGÚN ISO/IEC/IEEE 42010:2022

La norma ISO/IEC/IEEE 42010:2022 [20] no prescribe una arquitectura concreta. Establece qué debe contener una descripción arquitectónica y cómo se relacionan sus elementos. Su segunda edición cambia el sujeto de la descripción, que pasa de sistema de interés a entidad de interés; reemplaza el término marco de arquitectura por marco de descripción arquitectónica; sustituye los modelos de arquitectura por componentes de vista, y define dos conceptos que la edición de 2011 apenas mencionaba: las perspectivas de las partes interesadas, que agrupan preocupaciones, y los aspectos, que son características de la entidad que se reflejan en las vistas. La Tabla 7 muestra cómo se instanció cada concepto en este proyecto y en qué apartado se desarrolla.

Tabla 7. Instanciación de los conceptos de ISO/IEC/IEEE 42010:2022 en el proyecto

Concepto de la norma	Instanciación en el proyecto	Apartado
Entidad de interés	Sistema multi-agente de análisis de licitaciones públicas en su versión MVP, desplegado como una instancia por fábrica de software.	3.1
Entorno	SECOP II, proveedor de modelos de lenguaje, proveedor de identidad, servicio de correo electrónico y el marco de protección de datos personales de la Ley 1581 de 2012.	3.9
Partes interesadas	Diez roles, cada uno origen de al menos un escenario de la matriz ASR.	3.2
Perspectivas de las partes interesadas	Uso, negocio, seguridad y cumplimiento, y operación.	3.2
Preocupaciones	Diez preocupaciones (PR-01 a PR-10) trazadas a los escenarios ASR.	3.2

Concepto de la norma	Instanciación en el proyecto	Apartado
Aspectos	Desempeño, compatibilidad, fiabilidad, seguridad y flexibilidad, tomados de las características ISO/IEC 25010:2023 del árbol de utilidad.	3.3
Marco de descripción arquitectónica	Modelo C4 en sus niveles de contexto, contenedores y componentes, con diagramas complementarios dinámico y de despliegue.	3.8
Lenguaje de descripción arquitectónica	Structurizr DSL.	3.8
Puntos de vista y tipos de modelo	Cinco puntos de vista (PV-01 a PV-05).	3.8
Vistas y componentes de vista	Seis vistas generadas desde un único modelo de Structurizr.	3.9 a 3.13
Decisiones y justificación	Trece decisiones (DA-01 a DA-13) con alternativas descartadas y contrapartidas.	3.4 a 3.7
Correspondencias	Consistencia entre vistas por construcción del modelo y satisfacción de cada ASR por decisiones y elementos.	3.14

Fuente: elaboración propia.

3.2 PARTES INTERESADAS, PERSPECTIVAS Y PREOCUPACIONES

Las partes interesadas no se tomaron de una lista genérica. Cada una aparece en la matriz ASR como origen de al menos un escenario, de modo que es posible rastrear por qué su preocupación forma parte de la descripción. La Tabla 8 las agrupa en las cuatro perspectivas adoptadas, y la Tabla 9 enuncia las preocupaciones como preguntas que la arquitectura debe responder, junto con el escenario que las cuantifica.

Tabla 8. Partes interesadas agrupadas por perspectiva

Parte interesada	Interés en la arquitectura
Uso	
Analista Comercial	Obtener requisitos confiables sin leer el pliego completo y controlar qué se evalúa.
Gerencia y Comité Comercial	Recibir a tiempo una recomendación comparable entre licitaciones y sustentada en un informe verificable.
Negocio	
Gerencia Financiera	Mantener acotado el gasto en inferencia de los modelos de lenguaje.
Gerencia de Proyectos	Que la evaluación automática no reemplace la revisión humana de los requisitos.
Seguridad y cumplimiento	
Oficial de Seguridad de la Información	Proteger el portafolio, impedir accesos indebidos y resistir documentos manipulados.
Área de Calidad	Disponer de resultados íntegros y auditables.
Operación	
Administrador del Sistema	Recuperar el servicio y las tareas en curso después de una falla.
Equipo Técnico y Operaciones de TI	Aumentar la capacidad y desplegar sin modificar el código.
Arquitecto de Software	Mantener cada decisión trazable a un escenario y reversible cuando sea posible.
Estado colombiano (ANCP, SECOP II)	Que la información pública se consuma por los mecanismos autorizados y que el sistema tolere cambios en sus esquemas.

Fuente: elaboración propia.

Tabla 9. Preocupaciones arquitectónicas y escenarios que las cuantifican

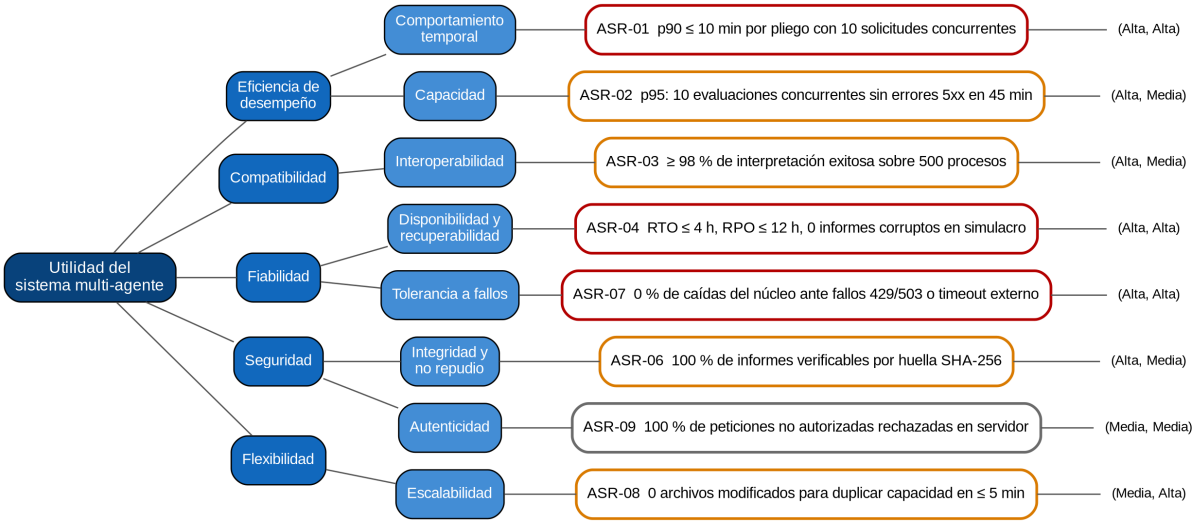
ID	Preocupación	Partes interesadas	ASR
PR-01	¿Termina el análisis de un pliego a tiempo para decidir, incluso con varias solicitudes simultáneas y sin bloquear al usuario?	Analista, Gerencia Comercial, Equipo Técnico	ASR-01, ASR-02, ASR-11
PR-02	¿Qué ocurre con un análisis en curso si cae la infraestructura o un servicio externo?	Administrador, Equipo Técnico	ASR-04, ASR-07
PR-03	¿Puede crecer la capacidad sin modificar el código?	Arquitecto, Operaciones de TI	ASR-08
PR-04	¿Sigue funcionando la ingesta cuando SECOP II cambia sus esquemas?	Analista, Estado colombiano	ASR-03
PR-05	¿Quién accede al portafolio y qué información sale hacia el proveedor del modelo?	Oficial de Seguridad, Dirección Comercial	ASR-05
PR-06	¿Puede alterarse un informe emitido sin que se detecte?	Área de Calidad, Gerencia Comercial	ASR-06
PR-07	¿Puede un usuario sin permiso, o con la sesión vencida, ejecutar operaciones protegidas?	Oficial de Seguridad	ASR-09
PR-08	¿Puede un pliego manipulado alterar el dictamen del agente?	Oficial de Seguridad, Arquitecto	ASR-10
PR-09	¿Se evalúa algún requisito sin aprobación humana previa?	Analista, Gerencia de Proyectos	ASR-12
PR-10	¿Quién limita el gasto en inferencia y qué ocurre cuando se agota el presupuesto?	Gerencia Financiera, Arquitecto	ASR-13

Fuente: elaboración propia.

3.3 IMPULSORES ARQUITECTÓNICOS: ÁRBOL DE UTILIDAD, RESTRICCIONES Y FUNCIONES CRÍTICAS

Las preocupaciones anteriores se cuantificaron como escenarios de calidad de seis partes (fuente, estímulo, entorno, artefacto, respuesta y medida de la respuesta) en la matriz ASR del Anexo B, y cada escenario se calificó en dos ejes: valor de negocio (VN) e impacto y dificultad arquitectónica (IA). De los quince escenarios evaluados, ocho son de tipo calidad con dictamen afirmativo, y solo ellos forman el árbol de utilidad de la Figura 1, porque son los únicos que admiten una comparación de prioridad entre sí.

Figura 1. Árbol de utilidad del sistema multi-agente



Nota: borde rojo, prioridad 1 (Alta, Alta); borde naranja, prioridad 2; borde gris, se documenta sin bloquear el diseño.
Fuente: elaboración propia a partir de la matriz ASR.
Para una lectura detallada, este diagrama se reproduce en formato ampliado en la Figura C.1 del Anexo C.

La lectura del árbol ordena el diseño. Los tres escenarios en (Alta, Alta) comparten un rasgo: el tiempo de respuesta de ASR-01, la recuperación de ASR-04 y la tolerancia a fallos de ASR-07 dependen de cómo se ejecuta una tarea larga que invoca servicios que el proyecto no controla. Por eso la primera decisión estructural fue definir dónde vive una tarea de análisis y cómo sobrevive a una falla; la tecnología se eligió después. Los escenarios de prioridad dos no cambian esa decisión, pero la restringen: ASR-08 exige que la respuesta a más carga sea añadir réplicas y no reescribir código, y ASR-06 obliga a que el resultado de cada tarea quede sellado en el momento de emitirse. ASR-09 queda en (Media, Media); se implementa con el mismo rigor, pero no condiciona la topología.

El árbol no agota los impulsores. Las restricciones y las funciones críticas no compiten por prioridad, se cumplen. La Tabla 10 las reúne junto con cuatro restricciones de proyecto que no provienen de un requisito del producto pero condicionan igualmente la solución.

Tabla 10. Restricciones y funciones críticas que condicionan la arquitectura

ID	Tipo	Enunciado resumido	Consecuencia para el diseño
ASR-05	Restricción	Confidencialidad del portafolio y de los datos estratégicos, en cumplimiento de la Ley 1581 de 2012.	Cifrado en reposo y en tránsito, custodia de claves y anonimización antes de toda llamada al proveedor del modelo.
ASR-13	Restricción	Techo de consumo de tokens por periodo.	Toda invocación al modelo pasa por un único punto de control.
ASR-10	Función crítica	El contenido de los pliegos se trata como dato no confiable.	Separar el canal de datos del canal de instrucciones del agente.
ASR-11	Función crítica	Análisis en segundo plano con identificador de tarea consultable.	Separar la recepción síncrona del procesamiento en unidades de despliegue distintas.

ID	Tipo	Enunciado resumido	Consecuencia para el diseño
ASR-12	Función crítica	El Agente Evaluador solo opera sobre requisitos aprobados por el analista.	Un orquestador con estado persistente que detenga el flujo hasta la aprobación.
RP-01	Restricción de proyecto	El producto está concebido para cualquier fábrica de software y no para una empresa en particular (Introducción).	Despliegue replicable por fábrica y núcleo que no dependa de un proveedor de nube concreto.
RP-02	Restricción de proyecto	Construcción iterativa con Scrum y prácticas DevSecOps sobre Azure DevOps (Diseño metodológico).	Unidades de despliegue que se construyan, analicen y prueben en un mismo pipeline.
RP-03	Restricción de proyecto	Alcance de Producto Mínimo Viable desarrollado en el marco de un trabajo de grado.	Preferir servicios administrados y reducir el número de piezas que operar.
RP-04	Restricción de proyecto	El frontend y el backend se desarrollan íntegramente en Python, por decisión del equipo del proyecto.	Un solo lenguaje en todas las unidades de despliegue; la interfaz web se construye con un marco de Python (DA-13).

Fuente: elaboración propia.

ASR-14 sigue pendiente de evidencia, pero no bloquea el diseño. La arquitectura depende de que las ponderaciones y umbrales del puntaje sean configurables y estén versionados (RF-032 y RF-033), no de los valores que finalmente apruebe la gerencia comercial. ASR-15 quedó fuera por no tener impacto estructural.

3.4 SELECCIÓN DEL ESTILO ARQUITECTÓNICO

Cada estilo arquitectónico favorece unos atributos de calidad a costa de otros [7], [27], así que la pregunta útil es cuál de ellos satisface estos impulsores con la menor complejidad operativa. Se compararon cinco estilos candidatos frente a los escenarios de la Figura 1, las funciones críticas y las restricciones de la Tabla 10. El resultado se resume en la Tabla 11 y se explica a continuación.

Tabla 11. Evaluación de estilos arquitectónicos frente a los impulsores

Impulsor	Monolito en capas	Microservicios	Orientado a eventos (coreografía)	Sin servidor (funciones)	Web-cola-trabajador con monolito modular
ASR-01 y ASR-02: desempeño bajo concurrencia	Parcial	Cumple	Cumple	Parcial	Cumple
ASR-04: recuperación de tareas en curso	No cumple	Cumple	Cumple	Parcial	Cumple
ASR-07: tolerancia a fallos externos	Parcial	Cumple	Cumple	Cumple	Cumple
ASR-08: escalar sin modificar código	No cumple	Cumple	Cumple	Cumple	Cumple
ASR-06 y ASR-09: integridad y autorización	Cumple	Parcial	Parcial	Parcial	Cumple
ASR-11: procesamiento en segundo plano	No cumple	Cumple	Cumple	Cumple	Cumple
ASR-12: compuerta de revisión humana	Cumple	Parcial	No cumple	Parcial	Cumple

Impulsor	Monolito en capas	Microservicios	Orientado a eventos (coreografía)	Sin servidor (funciones)	Web-cola-trabajador con monolito modular
ASR-13: control centralizado de cuota	Cumple	Parcial	Parcial	Parcial	Cumple
RP-01 a RP-03: complejidad operativa	Baja	Alta	Alta	Media	Media-baja
Resultado	Descartado	Descartado	Descartado	Descartado	Seleccionado

Fuente: elaboración propia.

Un monolito en capas desplegado como un solo proceso es la opción más sencilla de construir y operar, y en un dominio acotado sería la primera candidata. Aquí falla justo donde el árbol concentra la prioridad. Un análisis de hasta diez minutos consume recursos del mismo proceso que atiende a los usuarios, así que la interfaz y el procesamiento compiten entre sí, que es lo que ASR-11 prohíbe. Si el proceso cae, las tareas que estaban en memoria se pierden y ASR-04 no se cumple. Y para atender más análisis habría que replicar el sistema completo, incluida la parte que no necesita crecer. Ejecutar el análisis en hilos de fondo dentro del mismo proceso no resuelve ninguno de los tres problemas; solo los hace menos visibles.

Los microservicios satisfacen los escenarios de desempeño, fiabilidad y escalabilidad, pero lo hacen repartiendo el sistema en servicios con despliegue y datos propios, y ese reparto tiene un costo que el MVP no justifica. El sistema tiene una sola ruta con carga variable, la del análisis de pliegos. El resto del dominio como usuarios, portafolio, oportunidades, reglas y resultados atiende volúmenes bajos y evoluciona al mismo ritmo. Separarlo no añade capacidad donde se necesita y sí introduce llamadas por red entre módulos, consistencia eventual entre bases de datos y descubrimiento de servicios. Dos impulsores empeoran. ASR-06 exige que un informe, su puntaje y su huella queden sellados juntos, lo que entre bases de datos separadas obliga a coordinar transacciones distribuidas. La compuerta de ASR-12, además, quedaría repartida entre servicios. Los microservicios se registran como ruta de evolución si el producto llegara a operar con equipos independientes por dominio, condición que hoy no existe.

En una arquitectura orientada a eventos con coreografía, cada agente reacciona a lo que publica el anterior. El desacoplamiento es alto y el escalado sencillo, pero el control del flujo queda disperso y ningún componente es dueño de la secuencia completa. El Agente Evaluador se activaría al observar un evento de requisitos disponibles, y garantizar que ese evento solo exista después de la aprobación humana dependería de que todos los participantes respeten la convención. ASR-12 pide lo contrario: un punto único que haga imposible evaluar una lista no aprobada y que pueda demostrarse con una prueba. La traza de una tarea también se fragmenta entre suscriptores, lo que complica la bitácora de ASR-06 y la telemetría por tarea de RF-043. El proyecto conserva la mensajería, pero como cola de trabajo punto a punto y no como mecanismo de coordinación.

El estilo sin servidor ejecuta funciones por evento, escala a cero y reduce la operación. Su dificultad aquí es la duración y el estado. Las plataformas de funciones limitan el tiempo de

cada ejecución en sus planes de consumo, y un análisis cuyo percentil 90 admitido es de diez minutos queda demasiado cerca de esos límites para tratarlo como una sola invocación. Partirlo en funciones encadenadas exige un orquestador durable del proveedor que conserve el estado entre etapas y mantenga suspendida la tarea mientras el analista revisa los requisitos, una espera que puede durar horas. Esa pieza ata al proveedor de nube justo el componente que concentra ASR-04 y ASR-12, que es el que conviene mantener portable por RP-01.

El estilo web-cola-trabajador (web-queue-worker) separa un frontal web que atiende las solicitudes de un trabajador que ejecuta las tareas largas, y los comunica mediante una cola de mensajes [22]. Es casi literalmente la estructura que describe ASR-11: la API recibe la solicitud, encola el trabajo y responde de inmediato, y los trabajadores consumen la cola a su ritmo. La cola absorbe los picos de ASR-02, conserva los trabajos no confirmados si un trabajador cae (ASR-04) y permite añadir réplicas del trabajador sin tocar el código (ASR-08). La guía de referencia del estilo advierte que, sin un diseño cuidadoso, el frontal y el trabajador pueden crecer como bloques monolíticos difíciles de mantener [22]. Por eso la API y los trabajadores se construyen como un monolito modular: un solo código base con módulos de límites explícitos, desplegado en tres unidades independientes (API, trabajador de análisis y trabajador de vigilancia). A esas tres unidades se suma el servidor de interfaz web, también escrito en Python (DA-13), de modo que el sistema completo se despliega en cuatro unidades que comparten lenguaje, dependencias y cadena de construcción (RP-04). Se conserva la simplicidad de despliegue de un monolito y se obtiene la separación que exigen los escenarios en los puntos donde hace falta.

Dentro de cada unidad, las integraciones con servicios externos (SECOP II, proveedor del modelo, cola y almacenamiento) quedan detrás de puertos con adaptadores intercambiables. Esta estructura interna responde a dos impulsores distintos: ASR-03 necesita una capa anticorrupción que traduzca los esquemas de SECOP II antes de que lleguen al dominio, y RP-01 necesita que el núcleo no dependa de un proveedor de nube concreto.

3.5 PATRÓN DE COORDINACIÓN DE LOS AGENTES

Elegir el estilo del sistema no resuelve cómo colaboran los agentes dentro del trabajador. La literatura sobre sistemas multi-agente basados en modelos de lenguaje describe formas de organización que van desde flujos predefinidos hasta agentes que planifican sus propios pasos [2], [17], [21]. Anthropic propone una distinción útil para este caso: en un flujo de trabajo, los modelos y las herramientas se orquestan por rutas de código predefinidas, mientras que en un agente el modelo dirige de forma dinámica su propio proceso y el uso de herramientas [5]. La Tabla 12 compara cuatro patrones frente a los impulsores que afectan esta decisión.

Tabla 12. Patrones de coordinación de agentes evaluados

Patrón	Funcionamiento	Evaluación frente a los impulsores	Resultado
Agente autónomo con herramientas	Un modelo decide en cada paso qué herramienta usar hasta considerar terminada la tarea.	El número de pasos no está acotado, así que no hay garantía sobre el tiempo de ASR-01 ni sobre el techo de ASR-13. La aprobación humana de ASR-12 dependería de que el modelo decida detenerse.	Descartado
Conversación entre agentes	Varios agentes con roles intercambian mensajes hasta llegar a un resultado.	Cada ronda consume tokens y tiempo sin un límite estructural, y el resultado es difícil de reproducir, lo que choca con el versionado de RF-033 y con la verificación de RNF-006.	Descartado
Pizarra compartida	Los agentes leen y escriben sobre un espacio común y actúan cuando detectan información de su competencia.	Sirve cuando el orden de resolución no se conoce de antemano. Aquí la secuencia es fija y el control implícito debilita la compuerta de ASR-12.	Descartado
Orquestador con flujo predefinido y agentes especializados	Una máquina de estados fija la secuencia y delega cada etapa en un agente con una tarea acotada.	La secuencia es verificable, la espera humana es un estado explícito, el consumo por etapa está acotado y cada etapa se reanuda desde su checkpoint.	Seleccionado

Fuente: elaboración propia.

La decisión parte de una observación sobre el problema. La ruta para analizar un pliego es conocida y no cambia entre licitaciones; lo que varía es el contenido, y es ahí donde se usa el modelo de lenguaje. El orquestador implementa una máquina de estados con las etapas Recibida, Recuperando expediente, Saneando, Segmentando, Extrayendo, En espera de revisión, Evaluando, Calculando puntaje, Generando recomendación, Generando informe y Finalizada, más dos estados terminales: Fallida y Suspendida, este último para análisis detenidos por sospecha de inyección o por agotamiento de la cuota. Al cerrar cada etapa el estado se persiste como checkpoint. Cuando el flujo llega a En espera de revisión, el trabajador guarda el estado y libera la réplica. La aprobación del analista llega a la API, que encola un trabajo de reanudación, y cualquier réplica disponible continúa desde ese punto. La espera humana no ocupa capacidad de cómputo, y una caída a mitad del flujo solo obliga a repetir la etapa que estaba en curso. Este patrón corresponde a una orquestación centralizada con trabajadores especializados, del tipo que analizan Adimulam et al. [2], y se representa en C4 siguiendo la línea de Becattini et al. [8].

Tres reglas completan el patrón y responden a riesgos propios de los agentes. La primera es que los agentes no tienen herramientas con efectos: el Extractor y el Evaluador producen salidas estructuradas que se validan contra un esquema, y es el orquestador, que es código determinista, quien las aplica. Así se limita lo que puede lograr un pliego manipulado aunque consiga influir en el modelo, amenaza que OWASP ubica en el primer lugar de su clasificación de riesgos para aplicaciones con modelos de lenguaje [24]. La segunda es que el puntaje y la categoría de la recomendación se calculan con reglas versionadas y no con el modelo, que solo redacta la justificación a partir de los dictámenes emitidos; dos ejecuciones con las mismas entradas y la misma versión de reglas producen el mismo puntaje. La tercera es que ningún agente llama al proveedor directamente. Todas las invocaciones pasan por la pasarela de modelos de lenguaje, donde se aplican la anonimización, la cuota, los reintentos y la

telemetría.

3.6 DECISIONES ARQUITECTÓNICAS Y SU JUSTIFICACIÓN

Siguiendo el modelo conceptual de la norma, la descripción registra las decisiones que dan forma a la arquitectura junto con su justificación. Las decisiones de estilo y de patrón de los apartados 3.4 y 3.5 se completan aquí con las de tecnología y despliegue, que solo tienen sentido una vez fijados los dos primeros. La Tabla 13 registra cada decisión con los impulsores que la motivan y las alternativas descartadas. La Tabla 14 documenta sus contrapartidas: qué se gana, qué se cede y qué riesgo queda abierto con su mitigación. Esta segunda tabla corresponde a la hoja de tácticas y contrapartidas de la matriz ASR.

Tabla 13. Registro de decisiones arquitectónicas

ID	Decisión	Impulsores	Alternativas descartadas y motivo
DA-01	Estilo web-cola-trabajador con monolito modular desplegado en tres unidades.	ASR-01, ASR-02, ASR-04, ASR-08, ASR-11, RP-03	Monolito en capas, microservicios, coreografía de eventos y sin servidor (apartado 3.4).
DA-02	Puertos y adaptadores para toda integración externa, con capa anticorrupción para SECOP II.	ASR-03, RP-01	Acceso directo a las API externas desde el dominio: propaga los cambios de esquema y ata el núcleo al proveedor.
DA-03	Orquestador con máquina de estados persistente y agentes especializados, implementado con LangGraph y checkpoints en PostgreSQL.	ASR-01, ASR-04, ASR-12	Agente autónomo, conversación entre agentes y pizarra (apartado 3.5). Motores de flujo durable de propósito general (Temporal, Azure Durable Functions): añaden una plataforma más que operar o atan al proveedor de nube el componente más sensible.
DA-04	Cola durable administrada con confirmación explícita y cola de mensajes fallidos: Azure Service Bus.	ASR-04, ASR-07, ASR-08	RabbitMQ autoadministrado: garantías equivalentes, pero hay que operar y respaldar el intermediario. Redis como cola: su durabilidad depende de la configuración de persistencia.
DA-05	Una sola base PostgreSQL con pgvector para datos de negocio, checkpoints, índice vectorial y bitácora.	ASR-04, ASR-05, ASR-06, RP-03	Base vectorial ofrecida como servicio por un tercero: saca el portafolio del perímetro de la instancia. Base vectorial autoadministrada: otra pieza que operar sin necesidad para el portafolio de una sola fábrica.
DA-06	Documentos e informes en almacenamiento de objetos con política de inmutabilidad y huella SHA-256 encadenada en una bitácora de solo anexado.	ASR-06, RF-042	Guardar los PDF en la base de datos: crece el volumen de respaldos sin ganar integridad. Solo control de permisos: no detecta alteraciones hechas con credenciales válidas.
DA-07	Pasarela única hacia el proveedor del modelo con anonimización, cuota de tokens, reintentos con espera exponencial, disyuntor y telemetría.	ASR-05, ASR-07, ASR-13, RF-043	Llamadas directas desde cada agente: duplican controles y hacen imposible garantizar una cuota global.
DA-08	Guardia documental previa a la segmentación; agentes sin herramientas con efectos y salidas validadas contra esquema.	ASR-10	Confiar en que la instrucción del prompt haga ignorar el contenido malicioso: no es un control verificable.
DA-09	Puntaje y categoría de recomendación deterministas; el modelo solo redacta la justificación.	RF-033, RF-034, RNF-006	Puntaje estimado por el modelo: no es reproducible entre ejecuciones ni auditable por versión de reglas.
DA-10	Autenticación delegada con OpenID Connect y autorización basada en roles en la API, con verificación del estado del usuario en cada solicitud.	ASR-09, RNF-010	Autorización en la interfaz: se evita con peticiones directas. Tokens de larga duración sin verificación de estado: un usuario deshabilitado conserva acceso hasta que el token vence.

ID	Decisión	Impulsores	Alternativas descartadas y motivo
DA-11	Una instancia del sistema por fábrica de software (un solo inquilino).	ASR-05, RP-01	Plataforma multi-inquilino compartida: pondría en la misma base de datos los portafolios de fábricas que compiten por las mismas licitaciones.
DA-12	Despliegue de referencia en Azure con servicios administrados: Container Apps, Service Bus, PostgreSQL flexible, Blob Storage, Key Vault y Azure OpenAI.	ASR-04, ASR-05, ASR-08, RP-02, RP-03	Kubernetes administrado: control que el MVP no necesita a cambio de más operación. Máquinas virtuales: escalado y recuperación manuales. Otro proveedor de nube: viable técnicamente, pero separa el despliegue de la cadena de Azure DevOps y deja al proveedor del modelo fuera de la suscripción.
DA-13	Python en todas las unidades de despliegue: NiceGUI en la interfaz web, FastAPI en la API y LangGraph en los trabajadores.	RP-04, RP-02, RP-03, ASR-09	Aplicación de página única en React con TypeScript: añade un segundo lenguaje y una cadena de construcción de JavaScript, en contra de RP-04. Otros marcos de interfaz en Python (Streamlit, Reflex, Django con HTMX): se comparan en la Tabla 15.

Fuente: elaboración propia.

Tabla 14. Contrapartidas de las decisiones arquitectónicas

ID	Qué se gana	Qué se cede	Riesgo abierto y mitigación
DA-01	Carga síncrona y asíncrona separadas con escalado independiente, sobre un solo código base.	Los módulos comparten proceso dentro de cada unidad; un error en uno puede afectar a los demás.	Erosión de los límites entre módulos. Reglas de dependencia verificadas en el pipeline.
DA-02	Cambios de SECOP II o de proveedor confinados a un adaptador.	Una capa de traducción adicional con sus propias pruebas.	Adaptador desactualizado ante un cambio de esquema. Suite de integración con esquemas históricos (ASR-03).
DA-03	Secuencia verificable, espera humana sin consumo de cómputo y reanudación por etapa.	Dependencia de un marco de agentes y de su capa de abstracción; la durabilidad es por etapa, no por instrucción.	Etapas que dupliquen efectos al reanudarse: cada etapa escribe sus resultados con clave de tarea y etapa. Abstracción que oculta los prompts [5]: los prompts se mantienen explícitos y versionados en el repositorio.
DA-04	Entrega de al menos una vez, aislamiento de mensajes fallidos y escalado por longitud de cola.	Dependencia de un servicio de Azure.	Mensajes duplicados: consumidor idempotente por identificador de tarea. Portabilidad: el acceso pasa por un puerto (DA-02).
DA-05	Una sola pieza de datos que cifrar, respaldar y restaurar.	Búsqueda vectorial menos especializada que la de una base dedicada.	Degradación con índices muy grandes, fuera del volumen de una fábrica. Se revisa si el índice crece de forma sostenida.
DA-06	Alteraciones detectables e informes emitidos imposibles de sobrescribir.	Un informe emitido no se corrige; se emite uno nuevo con su propia huella.	Pérdida de la cadena de huellas. La bitácora entra en los respaldos y su verificación, en el plan de pruebas.
DA-07	Cuota, anonimización y resiliencia aplicadas en un solo lugar.	Todo el tráfico hacia el modelo pasa por un mismo componente.	Cuello de botella. La pasarela es un módulo dentro de cada réplica y no un servicio aparte, así que escala con el trabajador.
DA-08	Un pliego manipulado no puede ejecutar acciones ni saltarse la validación.	Agentes menos flexibles.	Inyecciones no detectadas que sesguen un dictamen. La revisión humana de ASR-12 y la suite adversaria de ASR-10 actúan como segunda barrera.
DA-09	Puntaje reproducible y auditable.	El puntaje no incorpora matices que el modelo podría percibir.	Justificación inconsistente con los dictámenes. Se genera solo desde dictámenes emitidos y se verifica en las pruebas de adecuación funcional.

ID	Qué se gana	Qué se cede	Riesgo abierto y mitigación
DA-10	Autorización que no se puede evitar desde el cliente y revocación inmediata.	Una consulta del estado del usuario por solicitud.	Latencia añadida. Consulta indexada con caché breve que se invalida al deshabilitar al usuario.
DA-11	Aislamiento completo de los datos de cada fábrica.	Cada fábrica despliega y actualiza su propia instancia.	Costo de operar varias instancias. Infraestructura como código y un mismo pipeline de despliegue.
DA-12	Menos piezas que operar; escalado y respaldo administrados.	Dependencia de Azure en el despliegue de referencia.	Traslado a otra nube. El núcleo corre en contenedores y usa puertos; cambian los adaptadores y la plantilla de infraestructura.
DA-13	Un solo lenguaje en todo el sistema; los tokens de acceso quedan en el servidor de interfaz y no llegan al navegador.	La interfaz mantiene una conexión persistente y estado en memoria por usuario; su catálogo de componentes es menor que el de React.	Pérdida de lo editado si se reinicia la réplica de interfaz: la revisión de requisitos se guarda como borrador en la API mientras se edita. Escalado de la interfaz: requiere afinidad de sesión, suficiente para los usuarios de un área comercial.

Fuente: elaboración propia.

Dos decisiones merecen un comentario adicional. DA-11 no aparece como requisito en el capítulo anterior, pero se deriva de dos hechos que sí están documentados: el producto se ofrece a cualquier fábrica de software y el portafolio que registra cada una es información estratégica protegida por ASR-05. Dos fábricas que usan el sistema pueden estar evaluando la misma licitación para competir entre sí, y ninguna medida de aislamiento lógico dentro de una base compartida elimina ese riesgo con la misma claridad que una instancia separada. DA-12, por su parte, fija un despliegue de referencia y no una dependencia del núcleo: el proyecto ya usa Azure DevOps (RP-02), y concentrar el MVP en servicios administrados del mismo proveedor reduce lo que hay que operar (RP-03), mientras que DA-02 mantiene la posibilidad de desplegar en otra nube sustituyendo adaptadores.

DA-13 se ajustó para cumplir RP-04, que exige construir el frontend y el backend en Python. En el servidor el cambio no tiene efecto, porque la API y los trabajadores ya estaban en ese lenguaje. En la interfaz obliga a sustituir la aplicación de página única en React por un marco web de Python, y la Tabla 15 compara los cuatro candidatos evaluados frente a las necesidades que imponen las historias de usuario y las decisiones anteriores.

Tabla 15. Evaluación de marcos de interfaz web en Python

Criterio	Streamlit	Reflex	Django con HTMX	NiceGUI
Interfaz escrita solo en Python (RP-04)	Cumple	Cumple	Parcial	Cumple
Formularios editables por fila para la revisión humana (RF-030)	Parcial	Cumple	Cumple	Cumple
Tablero filtrable y descarga de informes (RF-037, RF-038)	Cumple	Cumple	Cumple	Cumple
Consulta periódica del estado de tareas (RF-026)	Parcial	Cumple	Cumple	Cumple
Sin cadena de construcción de JavaScript	Cumple	No cumple	Cumple	Cumple

Criterio	Streamlit	Reflex	Django con HTMX	NiceGUI
Mismo ecosistema que la API (FastAPI)	No cumple	Parcial	No cumple	Cumple
Escalado horizontal de la interfaz	Parcial	Parcial	Cumple	Parcial
Resultado	Descartado	Descartado	Descartado	Seleccionado

Fuente: elaboración propia.

Streamlit se descartó por su modelo de ejecución: cada interacción vuelve a ejecutar el guion completo de la página, lo que complica los formularios con muchas filas editables que exige la revisión de requisitos y el manejo de varios roles con pantallas distintas. Reflex permite escribir la interfaz en Python, pero la compila a una aplicación React, de modo que la construcción depende de herramientas de JavaScript, y para compartir el estado entre varias réplicas recurre a un almacén externo como Redis, una pieza que el MVP no tiene. Django con HTMX es la opción más sólida para escalar, porque sus páginas se renderizan sin estado en el servidor, pero reparte la lógica de la interfaz entre plantillas HTML y Python e introduce un segundo marco web con su propio modelo de datos y de autenticación junto a FastAPI.

NiceGUI se seleccionó porque la interfaz se escribe por completo en Python, está construido sobre FastAPI, que es el mismo marco de la API [33], y ofrece de forma nativa los elementos que piden las historias de usuario: tablas interactivas para el tablero, formularios y diálogos para la revisión y un temporizador integrado para refrescar el estado de las tareas. En el navegador utiliza Vue y Quasar, pero ese código lo genera el propio marco y el equipo no lo escribe ni lo mantiene. Su contrapartida es que conserva en memoria el estado de cada usuario conectado, por lo que escalar la interfaz requiere afinidad de sesión. Se acepta porque la interfaz no está en la ruta crítica de carga, que es la del trabajador de análisis, y la atiende el número reducido de usuarios de un área comercial. Hay además una ganancia de seguridad: al operar como servidor, la interfaz actúa como cliente confidencial de OpenID Connect y los tokens de acceso nunca llegan al navegador, mientras que la autorización sigue resolviéndose en la API (DA-10).

3.7 CONSIDERACIONES EVALUADAS QUE NO APLICAN AL MVP

Dejar por escrito lo que se evaluó y no aplica evita que su ausencia se lea como un olvido. La Tabla 16 registra esas consideraciones junto con la condición que obligaría a revisarlas, de modo que la decisión de no adoptarlas pueda reabrirse con un criterio y no por intuición.

Tabla 16. Consideraciones evaluadas que no aplican al MVP

Consideración	Por qué no aplica	Condición que obligaría a revisarla
Plataforma multi-inquilino	Mezclaría datos estratégicos de fábricas que compiten entre sí (DA-11).	Oferta del producto como servicio compartido con aislamiento por inquilino verificable.
Microservicios por dominio	Hay una sola ruta con carga variable y el resto del dominio es de bajo volumen (3.4).	Equipos independientes por dominio o cargas dispares fuera del análisis.

Consideración	Por qué no aplica	Condición que obligaría a revisarla
CQRS y abastecimiento de eventos	La bitácora de solo anexado cubre la auditoría de ASR-06 sin reconstruir el estado a partir de eventos.	Necesidad de reconstruir cualquier estado histórico del dominio y no solo auditarlo.
Notificación del estado de tareas desde la API por WebSocket	El estado de una tarea cambia en intervalos de segundos a minutos y el sondeo de RF-026 basta. NiceGUI sí usa una conexión persistente, pero queda confinada entre el navegador y el servidor de interfaz; la API sigue sin estado.	Operaciones interactivas que requieran respuesta por debajo del segundo.
Caché distribuida	Con 10 evaluaciones concurrentes (RNF-002) la base de datos atiende las lecturas sin intermediarios.	Consultas del tablero que incumplan tiempos por volumen.
Alta disponibilidad multirregión	El tiempo de recuperación de 4 horas y la pérdida máxima de 12 horas de RNF-004 se atienden en una región con respaldo continuo.	Compromisos de disponibilidad superiores a los de una sola región.
Kubernetes administrado	Cuatro unidades de despliegue no justifican su operación (DA-12).	Decenas de servicios o requisitos de red que la plataforma de contenedores no cubra.
Modelos de lenguaje en infraestructura propia	Requiere GPU y operación especializada; la confidencialidad se atiende con anonimización y un proveedor dentro de la suscripción (ASR-05).	Política que prohíba enviar cualquier fragmento del portafolio fuera de la infraestructura de la fábrica.
Mensajería de publicación y suscripción	Cada tipo de trabajo tiene un solo consumidor; una cola punto a punto basta (3.4).	Varios consumidores independientes de un mismo evento.
Aplicación móvil nativa	Ninguna historia de usuario la requiere.	Una historia que exija operación sin conexión o notificaciones nativas.

Fuente: elaboración propia.

3.8 PUNTOS DE VISTA Y HERRAMIENTA DE MODELADO

Con las decisiones cerradas se definieron los puntos de vista que gobiernan las vistas de la descripción. En términos de la norma, un punto de vista establece las convenciones para construir, interpretar y usar una vista que atiende un conjunto de preocupaciones [20]. La Tabla 17 define cinco: los tres niveles del Modelo C4 que exige el objetivo, más un diagrama dinámico y uno de despliegue, que el propio Modelo C4 contempla como diagramas complementarios [9]. El nivel de código no se modela porque en un MVP ese detalle cambia en cada iteración y puede obtenerse del propio repositorio.

Tabla 17. Puntos de vista adoptados

ID	Punto de vista	Preocupaciones	Partes interesadas	Tipo de modelo	Vista
PV-01	Contexto	PR-04, PR-05	Todas	Diagrama de contexto C4	Figura 2
PV-02	Contenedores	PR-01, PR-02, PR-03, PR-06, PR-07	Arquitecto, Equipo Técnico, Oficial de Seguridad	Diagrama de contenedores C4	Figura 3
PV-03	Componentes	PR-05, PR-07, PR-08, PR-09, PR-10	Arquitecto, Equipo Técnico, Oficial de Seguridad	Diagramas de componentes C4	Figuras 4 y 5
PV-04	Comportamiento	PR-01, PR-02, PR-09	Analista, Gerencia de Proyectos, Arquitecto	Diagrama dinámico en secuencia	Figura 6

ID	Punto de vista	Preocupaciones	Partes interesadas	Tipo de modelo	Vista
PV-05	Despliegue	PR-02, PR-03, PR-05	Administrador, Operaciones de TI, Oficial de Seguridad	Diagrama de despliegue C4	Figura 7

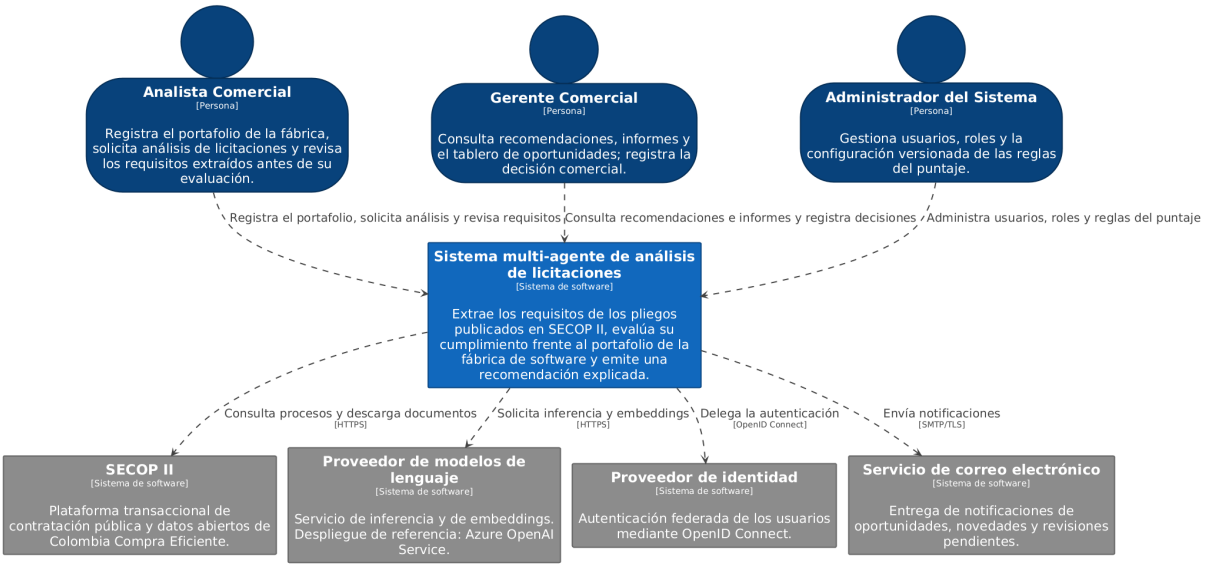
Fuente: elaboración propia.

Las vistas se construyeron con Structurizr, la herramienta creada por el autor del Modelo C4 [9]. La elección responde a la noción de correspondencia de la norma, que incluye relaciones de consistencia y de trazabilidad entre los elementos de una descripción [20]. En una herramienta de dibujo cada diagrama es un archivo independiente, y nada impide que un contenedor tenga un nombre en el nivel 2 y otro en el nivel 3, o que una relación aparezca en un diagrama y falte en otro. Structurizr parte de un modelo único escrito en su lenguaje específico de dominio (Structurizr DSL) y genera todas las vistas desde ese modelo, de modo que la coherencia entre ellas es una propiedad de la construcción y no de la disciplina de quien dibuja. El modelo es texto plano y se versiona en el mismo repositorio de Azure DevOps que el código, con lo que cada cambio arquitectónico queda revisable en el historial. Además permite registrar perspectivas por elemento, que se usaron para anotar qué escenarios ASR atiende cada contenedor. Como algunas vistas reúnen muchos elementos, el Anexo C las reproduce en páginas horizontales de mayor tamaño, a las que se llega desde el enlace ubicado bajo cada figura, y el Anexo D contiene el código completo con el que pueden regenerarse.

3.9 VISTA DE CONTEXTO (MODELO C4, NIVEL 1)

La Figura 2 delimita la entidad de interés y su entorno. Tres roles humanos usan el sistema: el Analista Comercial, que registra el portafolio y revisa los requisitos extraídos; el Gerente Comercial, que consume recomendaciones e informes y registra la decisión; y el Administrador del Sistema, que gestiona usuarios, roles y reglas del puntaje. El sistema depende de cuatro sistemas externos sobre los que el proyecto no tiene control: SECOP II, del que consulta procesos y descarga documentos; el proveedor de modelos de lenguaje, origen de ASR-07 y ASR-13; el proveedor de identidad, que evita que el sistema almacene contraseñas; y el servicio de correo, por el que salen las notificaciones.

Figura 2. Vista de contexto del sistema multi-agente (C4 Nivel 1)



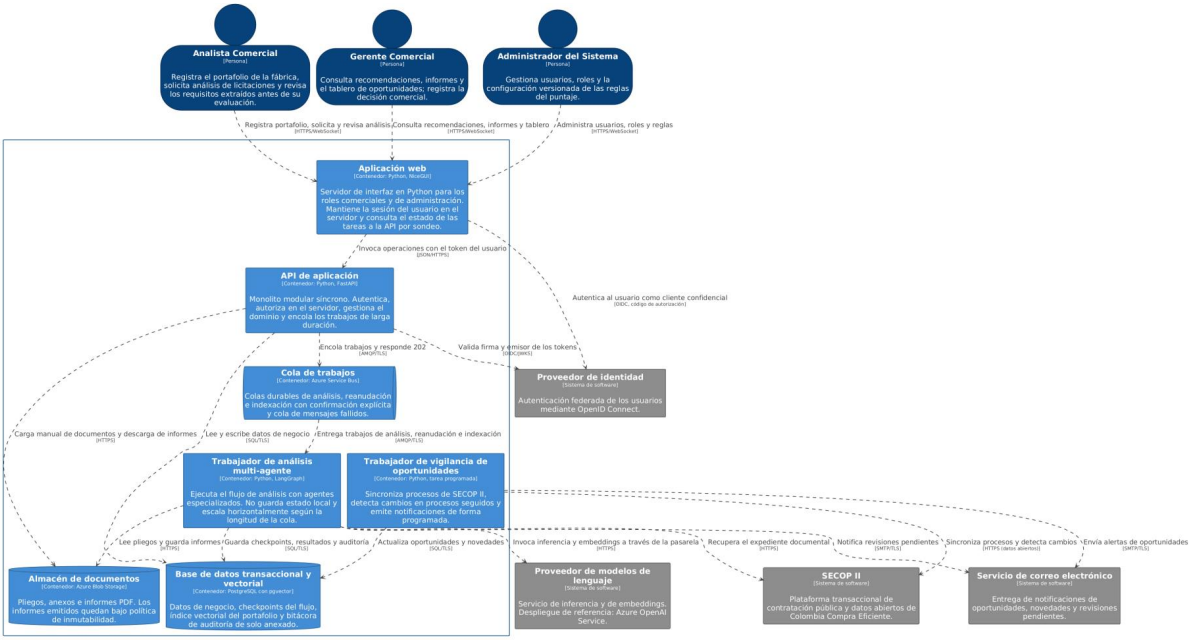
Fuente: elaboración propia con Structurizr.
Para una lectura detallada, este diagrama se reproduce en formato ampliado en la Figura C.2 del Anexo C.

La frontera tiene una restricción deliberada: el sistema solo consulta SECOP II y nunca presenta ofertas ni modifica información en la plataforma. La decisión de ofertar sigue siendo humana y queda registrada dentro del sistema (RF-039), lo que mantiene la recomendación automática en el lugar de apoyo a la decisión que le asigna el objetivo general.

3.10 VISTA DE CONTENEDORES (MODELO C4, NIVEL 2)

La Figura 3 muestra las unidades desplegables del sistema. El navegador del usuario solo se comunica con la aplicación web, un servidor de interfaz en Python que mantiene la sesión y reenvía cada operación a la API con el token del usuario; la API y el trabajador de análisis no se llaman entre sí y solo se comunican a través de la cola. La API y los trabajadores comparten la base de datos, algo que el estilo admite [22], y ese acoplamiento se controla con esquemas y permisos separados por módulo y con una bitácora que solo acepta inserciones. La Tabla 18 vincula cada contenedor con las decisiones y los escenarios que justifican su existencia.

Figura 3. Vista de contenedores del sistema multi-agente (C4 Nivel 2)



Fuente: elaboración propia con Structurizr.
Para una lectura detallada, este diagrama se reproduce en formato ampliado en la Figura C.3 del Anexo C.

Tabla 18. Contenedores del sistema y decisiones que los justifican

Contenedor	Tecnología	Responsabilidad	Decisiones y escenarios
Aplicación web	Python, NiceGUI	Servidor de interfaz de los tres roles; mantiene la sesión en el servidor, consulta el estado de las tareas por sondeo y no toma decisiones de autorización.	DA-13, DA-10, RP-04
API de aplicación	Python, FastAPI	Autenticación, autorización, lógica del dominio y encolado de trabajos con respuesta 202.	DA-01, DA-10, ASR-09, ASR-11
Cola de trabajos	Azure Service Bus	Trabajos de análisis, reanudación e indexación con cola de mensajes fallidos.	DA-04, ASR-04, ASR-07, ASR-08
Trabajador de análisis multi-agente	Python, LangGraph	Flujo de análisis, agentes especializados, puntaje, recomendación e informe.	DA-03, DA-07, DA-08, DA-09, ASR-01, ASR-10, ASR-12, ASR-13
Trabajador de vigilancia de oportunidades	Python, tarea programada	Sincronización con SECOP II, detección de cambios y alertas.	DA-02, ASR-03, ASR-07
Base de datos transaccional y vectorial	PostgreSQL con pgvector	Datos de negocio, checkpoints, índice vectorial del portafolio y bitácora de auditoría.	DA-05, ASR-04, ASR-05, ASR-06
Almacén de documentos	Azure Blob Storage	Pliegos, anexos e informes emitidos bajo política de inmutabilidad.	DA-06, ASR-06

Fuente: elaboración propia.

El trabajador de vigilancia se separa del de análisis aunque ambos comparten código. Sus cargas tienen naturaleza distinta, una periódica y predecible y otra bajo demanda, y una falla prolongada de SECOP II durante la sincronización no debe consumir las réplicas que atienden

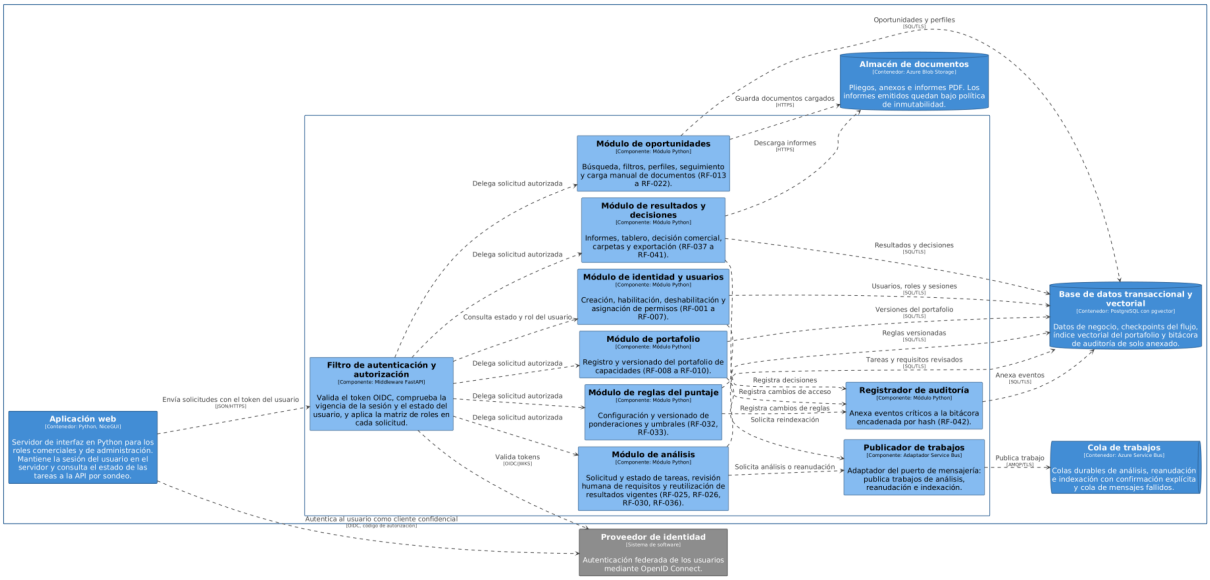
análisis cuyo expediente ya fue recuperado (ASR-07).

3.11 VISTAS DE COMPONENTES (MODELO C4, NIVEL 3)

El nivel 3 se desarrolló para los dos contenedores donde se concentran las decisiones: la API, porque en ella se resuelve la autorización de ASR-09, y el trabajador de análisis, porque reúne las tres funciones críticas y las dos restricciones. Los demás contenedores son servicios administrados o tienen una responsabilidad única que el nivel 2 ya describe, y descomponerlos no añadiría información para ninguna preocupación.

En la API (Figura 4), toda solicitud entra por el filtro de autenticación y autorización, que valida el token contra el proveedor de identidad, consulta el estado y el rol del usuario y solo entonces delega en el módulo correspondiente. Como no existe otra ruta hacia los módulos, una petición construida a mano fuera de la interfaz recibe el mismo tratamiento que una hecha desde ella (ASR-09), y un usuario deshabilitado pierde el acceso en la solicitud siguiente (RNF-010). Los módulos coinciden con los bloques temáticos de requisitos del capítulo anterior, y solo los de análisis y portafolio publican trabajos en la cola.

Figura 4. Componentes de la API de aplicación (C4 Nivel 3)

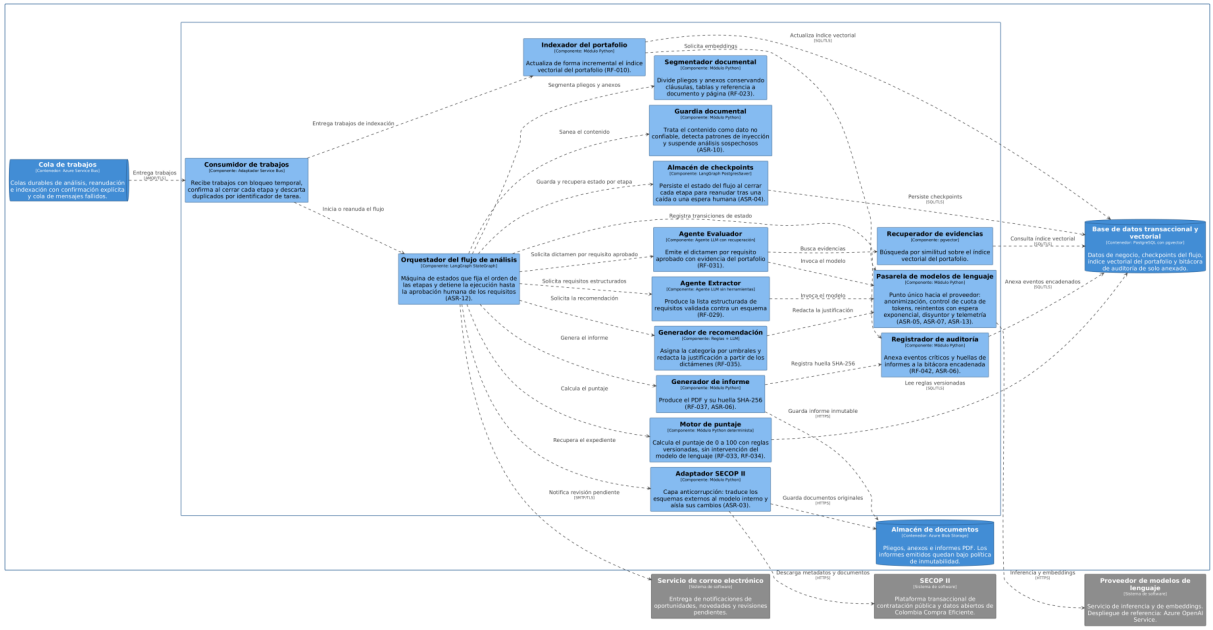


Fuente: elaboración propia con Structurizr.
Para una lectura detallada, este diagrama se reproduce en formato ampliado en la Figura C.4 del Anexo C.

En el trabajador de análisis (Figura 5), el orquestador es el único componente que conoce la secuencia. El guardia documental actúa antes de la segmentación, de modo que ningún agente recibe contenido sin sanear (ASR-10). El Extractor y el Evaluador solo se comunican con el exterior a través de la pasarela de modelos de lenguaje, que concentra la anonimización de ASR-05, la resiliencia de ASR-07 y la cuota de ASR-13. El motor de puntaje no tiene relación con la pasarela: su única dependencia son las reglas versionadas en la base de datos, y esa ausencia en el diagrama es la evidencia visual de DA-09. El generador de informe entrega la

huella SHA-256 al registrador de auditoría, que la anexa a la bitácora encadenada (ASR-06).

Figura 5. Componentes del trabajador de análisis multi-agente (C4 Nivel 3)

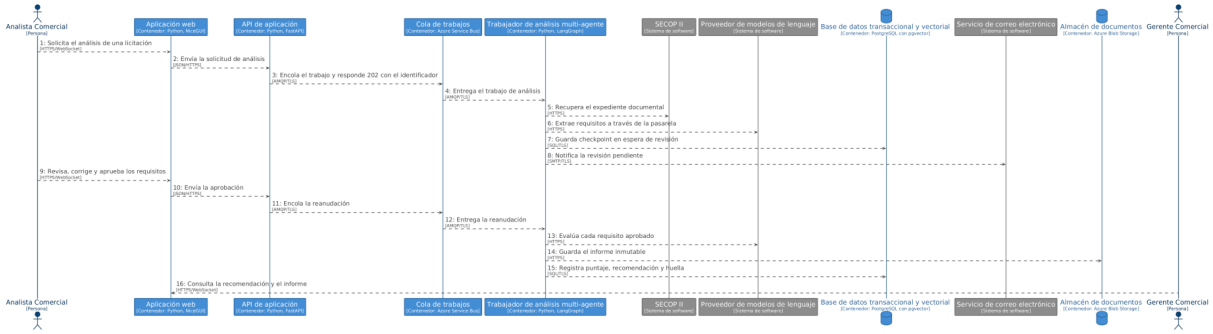


Fuente: elaboración propia con Structurizr.
Para una lectura detallada, este diagrama se reproduce en formato ampliado en la Figura C.5 del Anexo C.

3.12 VISTA DINÁMICA DEL FLUJO DE ANÁLISIS

La Figura 6 recorre un análisis completo e ilustra en el tiempo tres decisiones que las vistas estáticas solo pueden sugerir. En el paso 3 la API responde con el identificador de tarea antes de que empiece cualquier procesamiento (ASR-11). Entre los pasos 7 y 12 el flujo se detiene: el trabajador guarda el checkpoint, notifica al analista y libera la réplica; la aprobación entra por la API, donde se autoriza, y la reanudación puede tomarla cualquier réplica disponible (ASR-12 y ASR-04). En los pasos 14 y 15 el informe se guarda como objeto inmutable y su huella queda en la bitácora antes de que el gerente pueda consultarlo (ASR-06).

Figura 6. Flujo de análisis con revisión humana en el ciclo (vista dinámica)

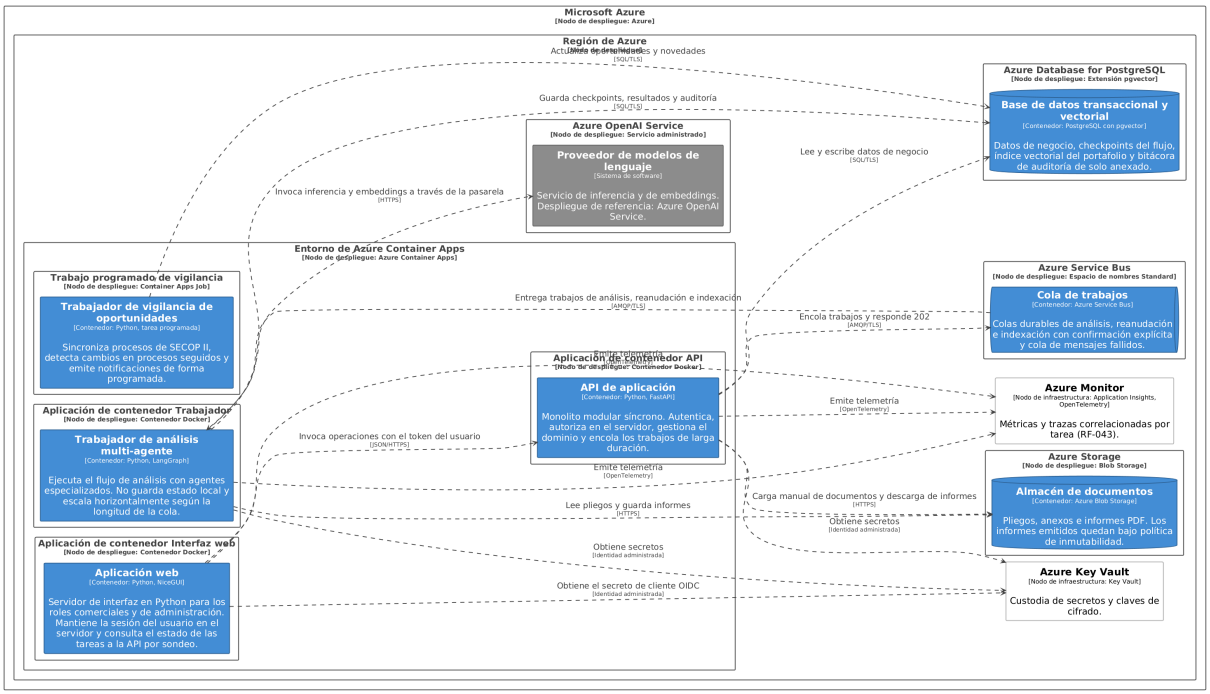


Fuente: elaboración propia con Structurizr.
Para una lectura detallada, este diagrama se reproduce en formato ampliado en la Figura C.6 del Anexo C.

3.13 VISTA DE DESPLIEGUE

La Figura 7 muestra el despliegue de referencia del MVP. Cada fábrica de software dispone de su propia suscripción (DA-11). La aplicación web, la API, el trabajador de análisis y el trabajador de vigilancia corren como contenedores Docker en un entorno de Azure Container Apps dentro de una red virtual privada. La aplicación web se configura con afinidad de sesión, porque NiceGUI conserva en memoria el estado de cada usuario conectado. El trabajador de análisis escala entre cero y N réplicas según la longitud de la cola, mediante reglas de escalado basadas en KEDA, lo que atiende ASR-08 sin modificar código; si el arranque en frío de la primera réplica llegara a comprometer el percentil 90 de ASR-01, se fija una réplica mínima activa. El trabajador de vigilancia se ejecuta como trabajo programado.

Figura 7. Despliegue de referencia del MVP en Azure



Fuente: elaboración propia con Structurizr.
Para una lectura detallada, este diagrama se reproduce en formato ampliado en la Figura C.7 del Anexo C.

Los datos se reparten en tres servicios administrados. Azure Database for PostgreSQL, en su modalidad de servidor flexible con la extensión pgvector, aporta el respaldo continuo con restauración a un punto en el tiempo que sostiene ASR-04. Azure Blob Storage conserva pliegos e informes con política de inmutabilidad. Azure Key Vault custodia secretos y claves, a los que la API y los trabajadores acceden con identidades administradas, sin credenciales en el código ni en la configuración (ASR-05). La telemetría correlacionada por tarea de RF-043 se envía a Azure Monitor mediante OpenTelemetry, y el proveedor de modelos de lenguaje de referencia, Azure OpenAI Service, se despliega en la misma suscripción. La región queda pendiente, porque depende de la política de transferencia internacional de datos que regula la Ley 1581 de 2012.

3.14 CORRESPONDENCIAS Y TRAZABILIDAD

La norma define las correspondencias como relaciones identificadas entre elementos de la descripción, entre ellas la consistencia, la trazabilidad y la satisfacción [20]. Esta descripción usa dos. La consistencia entre vistas la asegura el modelo único de Structurizr, porque un elemento existe una sola vez y todas las vistas lo referencian. La satisfacción de los impulsores se documenta en la Tabla 19, que vincula cada ASR aprobado con las decisiones que lo atienden, los elementos donde se materializa, las vistas donde se evidencia y la prueba con que se verificará en el cuarto objetivo específico.

Tabla 19. Correspondencia entre escenarios ASR, decisiones, elementos y vistas

ASR	Decisiones	Elementos que lo satisfacen	Vistas	Verificación prevista
ASR-01	DA-01, DA-03, DA-13	Trabajador de análisis con procesamiento paralelo de fragmentos; cola de trabajos.	Fig. 3, 5, 6	Prueba de carga con 10 solicitudes concurrentes durante 60 minutos.
ASR-02	DA-01, DA-04	Cola de trabajos; réplicas del trabajador.	Fig. 3, 7	Prueba de estrés sostenida durante 45 minutos.
ASR-03	DA-02	Adaptador SECOP II; trabajador de vigilancia.	Fig. 3, 5	Integración contra 500 esquemas históricos de SECOP II.
ASR-04	DA-03, DA-04, DA-05	Almacén de checkpoints; cola de trabajos; PostgreSQL con respaldo continuo.	Fig. 5, 6, 7	Simulacro de caída forzada durante un análisis.
ASR-05	DA-05, DA-07, DA-11, DA-12	Pasarela con anonimización; Key Vault; cifrado de base de datos y almacén.	Fig. 5, 7	Pruebas SAST y DAST e inspección de cifrado en el pipeline.
ASR-06	DA-06	Generador de informe; registrador de auditoría; almacén de documentos inmutable.	Fig. 5, 6	Manipulación directa en base de datos y detección por huella.
ASR-07	DA-04, DA-07	Pasarela con reintentos y disyuntor; cola con mensajes fallidos; trabajador de vigilancia separado.	Fig. 3, 5	Inyección controlada de errores 429, 503 y tiempos de espera.
ASR-08	DA-01, DA-12	Trabajador sin estado; escalado por longitud de cola.	Fig. 3, 7	Despliegue de dos réplicas adicionales sin cambios de código.
ASR-09	DA-10	Filtro de autenticación y autorización; módulo de identidad.	Fig. 4	Suite de 200 pruebas rol-operación sobre los endpoints protegidos.
ASR-10	DA-08	Guardia documental; agentes sin herramientas con efectos.	Fig. 5	Suite adversaria de 50 pliegos manipulados.
ASR-11	DA-01	API de aplicación; cola de trabajos.	Fig. 3, 6	Respuesta 202 en menos de 1 s y consulta periódica del progreso.
ASR-12	DA-03	Orquestador del flujo; módulo de análisis.	Fig. 4, 5, 6	Intento de forzar la evaluación de una lista no aprobada.
ASR-13	DA-07	Pasarela de modelos de lenguaje.	Fig. 5	Cuota simulada agotada y verificación del bloqueo inmediato.

Fuente: elaboración propia.

Una regla de correspondencia gobierna la tabla: todo ASR con dictamen afirmativo debe quedar satisfecho por al menos un elemento, y todo contenedor debe responder por al menos una decisión o un escenario. Las trece filas de la Tabla 19 y las siete de la Tabla 18 muestran que ambas condiciones se cumplen.

3.15 RESULTADO PARCIAL DEL OBJETIVO Y PENDIENTES DECLARADOS

Con este capítulo queda definida la arquitectura del MVP: un estilo web-cola-trabajador construido como monolito modular en tres unidades de despliegue más un servidor de interfaz, todo en Python, con un orquestador de flujo predefinido que coordina agentes especializados y detiene la ejecución hasta la revisión humana. La descripción contiene los elementos del modelo conceptual de la norma ISO/IEC/IEEE 42010:2022: la entidad de interés y su entorno, diez partes interesadas en cuatro perspectivas, diez preocupaciones, cinco puntos de vista, seis vistas generadas desde un modelo único, trece decisiones con sus alternativas y contrapartidas, y una tabla de correspondencias en la que los trece ASR aprobados quedan satisfechos.

Quedan pendientes que no alteran el estilo ni el patrón, pero deben cerrarse antes de la implementación. La matriz de roles y permisos, heredada del primer objetivo, es necesaria para que el filtro de autorización opere. Las ponderaciones y umbrales de ASR-14 son la configuración que consume el motor de puntaje. La región de Azure depende de la política de transferencia internacional de datos. Y la selección concreta del proveedor de identidad y de los modelos de lenguaje se hará en el tercer objetivo, con pruebas comparativas sobre pliegos reales y a través de los puertos que la arquitectura ya prevé.

Con las decisiones de arquitectura justificadas, el desarrollo de este objetivo continúa con el diseño de la interfaz de usuario. Ese diseño parte de tres entradas definidas aquí: los roles de la vista de contexto, las historias de usuario de la Tabla 3 y dos restricciones que la arquitectura impone a la interfaz, que ninguna decisión de autorización se toma en el cliente y que el estado de las tareas se consulta por sondeo. Como la interfaz se construye con NiceGUI, las maquetas se diseñarán sobre los componentes que ofrece ese marco.

4. DESARROLLO DEL TERCER OBJETIVO

(Usted define el nombre del capítulo según su objetivo)

Aquí explica todo el desarrollo del objetivo, el resultado y su conclusión

6. CONCLUSIONES

Son párrafos cortos en donde se generalizan los resultados al lector. Recogen información que da respuesta a la pregunta problema, al cumplimiento de los objetivos y a las temáticas abordadas a lo largo del trabajo de grado.

Las conclusiones se deben escribir en pasado.

7. RECOMENDACIONES

Las recomendaciones se debe relacionar los posibles alcances al trabajo de grado desarrollado, haciendo relación a variables o problemas que no se abarcaron en este trabajo.

Al igual relacionar si es pertinente los trabajos futuros sobre la misma temática

BIBLIOGRAFÍA

1. ACHIAM, Josh; ADLER, Steven; AGARWAL, Sandhini *et al.* GPT-4 Technical Report. *OpenAI*. 2023. [en línea]. Disponible en: https://arxiv.org/abs/2303.08774 [Consulta: 15 marzo 2026].
2. ADIMULAM, Aditya; GUPTA, Rahul; KUMAR, Sanjay. The Orchestration of Multi-Agent Systems: Architectures, Protocols, and Enterprise Adoption. 2025. [en línea]. Disponible en: https://arxiv.org/abs/2601.13671 [Consulta: 15 marzo 2026].
3. AMALFITANO, Domenico; DISTANTE, Damiano; MATALONGA, Santiago; FARALLI, Stefano; HAUCK, Jean Carlo Rossa. Artificial Intelligence Applied to Software Testing: A Tertiary Study. *ACM Computing Surveys*. 2023, 56(3). [en línea]. Disponible en: https://doi.org/10.1145/3616372
4. AMARO, Rúben; PEREIRA, Ruben; DA SILVA, Miguel Mira. DevSecOps practices and tools. *International Journal of Information Security*. 2024. [en línea]. Disponible en: https://doi.org/10.1007/s10207-024-00914-z
5. ANTHROPIC. Building effective agents. *Anthropic Engineering*. 2024. [en línea]. Disponible en: https://www.anthropic.com/engineering/building-effective-agents [Consulta: 17 septiembre 2026].
6. ARSLAN, Mehmet; MUNAWAR, Sadia; CRUZ, Christophe. Sustainable Digitalization of Business with Multi-Agent RAG and LLM. *Procedia Computer Science, KES 2024*. 2024. [en línea]. Disponible en: https://arxiv.org/abs/2502.15700 [Consulta: 15 marzo 2026].
7. BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software Architecture in Practice. 4th ed. Boston: *Addison-Wesley*, 2022. ISBN 978-0136886099.
8. BECATTINI, Mirco; VERDECCHIA, Roberto; VICARIO, Enrico *et al.* Describing Agentic AI Systems with C4: Lessons from Industry Projects. En: *IEEE/ACM SATrends Workshop*. 2025. [en línea]. Disponible en: https://arxiv.org/abs/2603.15021 [Consulta: 15 marzo 2026].
9. BROWN, Simon. The C4 model for visualising software architecture. [en línea]. Disponible en: https://c4model.com [Consulta: 17 septiembre 2026].
10. BUTT, Shariq; WEN-LI, Wang *et al.* A systematic review of machine learning methods in software testing. *Applied Soft Computing*. 2024. [en línea]. Disponible en: https://doi.org/10.1016/j.asoc.2024.111805
11. COLOMBIA COMPRA EFICIENTE — AGENCIA NACIONAL DE CONTRATACIÓN PÚBLICA. SECOP II: estadísticas de contratación pública digital en Colombia, 2025. Bogotá: ANCP, 2025. [en línea]. Disponible en: https://www.colombiacompra.gov.co/secop/secop-ii [Consulta: 15 marzo 2026].
12. FERRARI, Alessio; SPOLETINI, Paola; AZZEH, Mohammad *et al.* Large Language Models for Requirements Engineering: A Systematic Literature Review. 2025. [en línea]. Disponible en: https://arxiv.org/abs/2509.11446 [Consulta: 15 marzo 2026].
13. FIDA, Memoona; SIDDIQUI, Adeel Baig; QADIR, Muhammad Abdul *et al.* Explainable Artificial Intelligence-Based Decision Support Systems: A Recent Review. *Electronics*. 2024, 13(14), 2842. [en línea]. Disponible en: https://doi.org/10.3390/electronics13142842
14. FU, Michael; PASUKSMIT, Jirat; TANTITHAMTHAVORN, Chakkrit. AI for DevSecOps: A Landscape and Future Opportunities. *ACM Transactions on Software Engineering and Methodology*. 2024. [en línea]. Disponible en: https://doi.org/10.1145/3712190
15. GRUPO XAI/PROCUREMENT. From LLM Predicates to Logic Tensor Networks:

Neurosymbolic Offer Validation in Regulated Procurement. 2025. [en línea]. Disponible en: https://arxiv.org/abs/2604.05539 [Consulta: 15 marzo 2026].
16. GUIDA, Matteo; CANIATO, Federico; MORETTO, Antonella; RONCHI, Stefano. The role of artificial intelligence in the procurement process: State of the art and research agenda. *Journal of Purchasing and Supply Management*. 2023, 29(2), 100823. [en línea]. Disponible en: https://doi.org/10.1016/j.pursup.2023.100823
17. GUO, Taicheng; CHEN, Xiuying; WANG, Yaqi *et al.* Large Language Model Based Multi-Agents: A Survey of Progress and Challenges. 2024. [en línea]. Disponible en: https://arxiv.org/abs/2402.01680 [Consulta: 15 marzo 2026].
18. HE, Junda; TREUDE, Christoph; LO, David. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. *ACM Transactions on Software Engineering and Methodology*. 2025. [en línea]. Disponible en: https://doi.org/10.1145/3712347
19. HONG, Sirui; ZHUGE, Mingchen; CHEN, Jonathan; ZHENG, Xiawu *et al.* MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. En: *Proceedings of ICLR 2024*. 2024. [en línea]. Disponible en: https://arxiv.org/abs/2308.00352 [Consulta: 15 marzo 2026].
20. IEEE; ISO; IEC. ISO/IEC/IEEE 42010:2022. Software, systems and enterprise — Architecture description. New York: *IEEE*, 2022. [en línea]. Disponible en: https://doi.org/10.1109/IEEESTD.2022.9938446 [Consulta: 15 marzo 2026].
21. LI, Shengwei; ZHAO, Yue; SHI, Chuan; LIU, Bo *et al.* A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth / Springer Nature*. 2024. [en línea]. Disponible en: https://doi.org/10.1007/s44336-024-00009-2
22. MICROSOFT. Web-Queue-Worker architecture style. *Azure Architecture Center*. [en línea]. Disponible en: https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/web-queue-worker [Consulta: 17 septiembre 2026].
23. NAI, Roberto; SULIS, Emilio; MEO, Rosa. Public tenders, complaints, machine learning and recommender systems: a case study in public administration. *Computer Law & Security Review*. 2023. [en línea]. Disponible en: https://doi.org/10.1016/j.clsr.2023.105882
24. OWASP FOUNDATION. OWASP Top 10 for LLM Applications 2025. *OWASP GenAI Security Project*. 2024. [en línea]. Disponible en: https://genai.owasp.org/llm-top-10/ [Consulta: 17 septiembre 2026].
25. QASEM, Mohammad H.; HUDAIB, Amjad; OBEID, Nadim; ALMAIAH, Mohammed A. *et al.* An Intelligent Decision Support System Based on Multi Agent Systems for Business Classification Problem. *Sustainability*. 2023, 15(14), 10977. [en línea]. Disponible en: https://doi.org/10.3390/su151410977
26. RAJAPAKSE, Roshan Namal; ZAHEDI, Mansooreh; BABAR, M. Ali; SHEN, Haifeng. Challenges of Adopting DevSecOps in the Software Industry: A Systematic Review. *ACM Computing Surveys*. 2022, 55(12). [en línea]. Disponible en: https://doi.org/10.1145/3539856
27. RICHARDS, Mark; FORD, Neal. Fundamentals of Software Architecture: An Engineering Approach. Sebastopol: *O'Reilly Media*, 2020. ISBN 978-1492043454.
28. SICILIANI, Lucia; TACCARDI, Vito; BASILE, Pierpaolo; DI CIANO, Mauro; LOPS, Pasquale. AI-based decision support system for public procurement. *Information Systems*. 2023, 119, 102284. [en línea]. Disponible en:

https://doi.org/10.1016/j.is.2023.102284
29. SOFAT, Ananya; SODHI, Balwinder. Speeding up Government Procurement Workflows with LLMs. En: *EGOVIS 2024, LNCS 14913*. Springer, 2024, pp. 33–47. [en línea]. Disponible en: https://doi.org/10.1007/978-3-031-68211-7_3
30. TAKAYAMA, Yuta; YOSHIZAWA, Akira; SHINDO, Hiroyuki *et al.* Tender Document Analyzer with the Combination of Supervised Learning and LLM-based Improver. En: *WWW '24 Companion*. ACM, 2024. [en línea]. Disponible en: https://doi.org/10.1145/3589335.3651233
31. WANG, Lei; MA, Chen; FENG, Xueyang; ZHANG, Zeyu *et al.* A Survey on Large Language Model based Autonomous Agents. *Frontiers of Computer Science*. 2024, 18, 186345. [en línea]. Disponible en: https://doi.org/10.1007/s11704-024-40231-1
32. WASEEM, Muhammad; MIKKONEN, Tommi; LEPPÄNEN, Marko; HYRYNSALMI, Sami *et al.* Artificial Intelligence Procurement Assistant: Enhancing Bid Evaluation. En: *International Conference on Software Business, LNBIP 500*. Springer, 2024, pp. 113–127. [en línea]. Disponible en: https://doi.org/10.1007/978-3-031-53227-6_8
33. ZAUBERZEUG GMBH. NiceGUI: Create web-based user interfaces with Python. [en línea]. Disponible en: https://nicegui.io [Consulta: 21 septiembre 2026].
34. ZHANG, Jie Ming; HARMAN, Mark; MA, Lei; LIU, Yang. Machine Learning Testing: Survey, Landscapes and Horizons. *IEEE Transactions on Software Engineering*. 2022, 48(2), 1–36. [en línea]. Disponible en: https://doi.org/10.1109/TSE.2019.2962027
35. ZHAO, Yue; LI, Dan. A Large Language Model-based Framework for Semi-Structured Tender Document Retrieval-Augmented Generation. 2024. [en línea]. Disponible en: https://arxiv.org/abs/2410.09077 [Consulta: 15 marzo 2026].
36. ZIEGLER, Albert; EBERT, Christof; TRENDOWICZ, Adam. Infusing Artificial Intelligence Into Software Engineering and the DevSecOps Continuum. *IEEE Computer*. 2024, 57. [en línea]. Disponible en: https://doi.org/10.1109/MC.2024.3423108

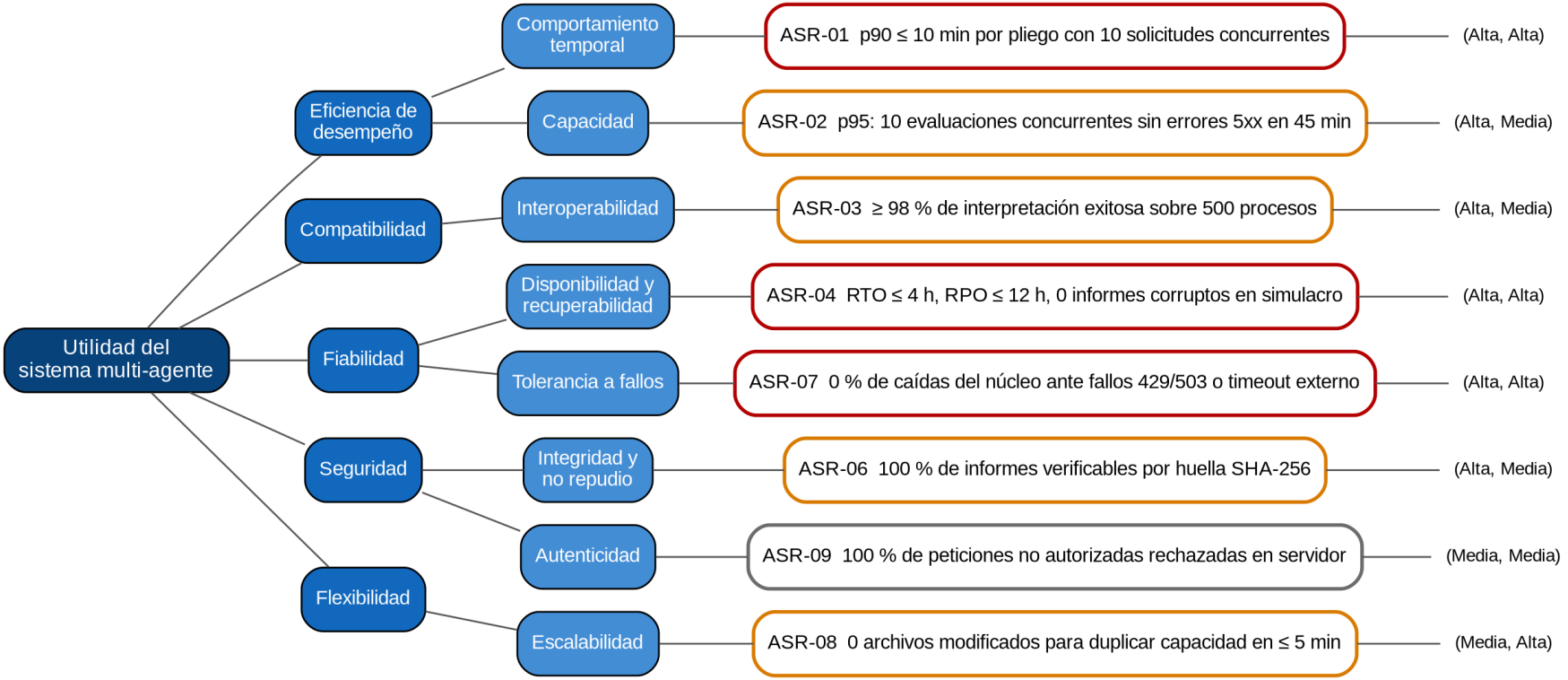
ANEXOS

Anexo A. Matriz de Requisitos: Matriz_Requisitos_SistemaMultiAgente.xlsx
Anexo B. Matriz ASR:Matriz_ASR_UtilityTree_SistemaMultiAgente.xlsx

Anexo C. Diagramas de arquitectura en formato ampliado

Este anexo reproduce, en páginas horizontales y a mayor tamaño, los diagramas del capítulo 3. Cada figura conserva la numeración del cuerpo del documento y enlaza de vuelta al apartado donde se discute. Las imágenes se insertaron en su resolución original, por lo que en la versión digital del documento pueden ampliarse con el zoom del lector sin perder legibilidad. Las Figuras C.2 a C.7 se generaron desde el espacio de trabajo de Structurizr incluido en el Anexo D.

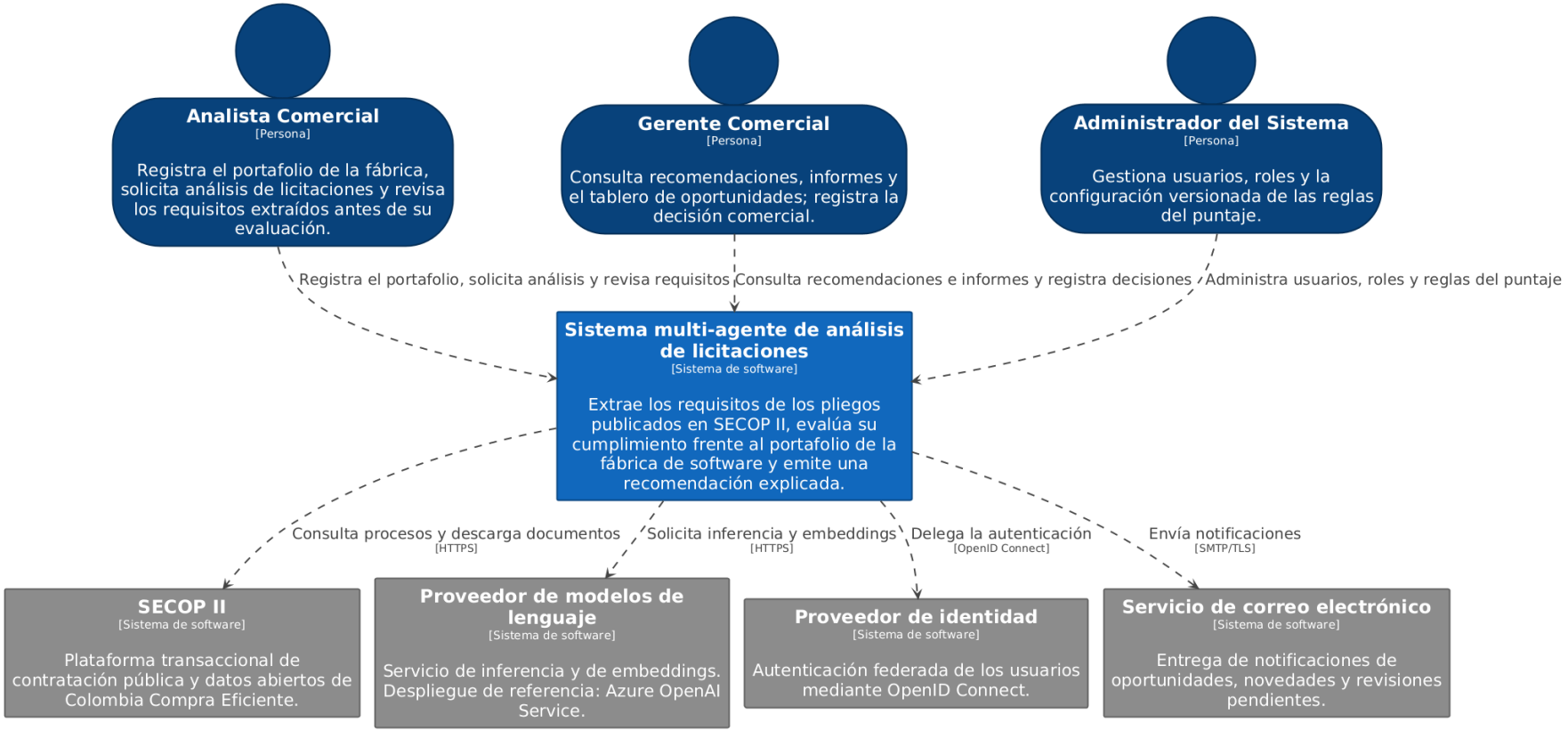
Figura C.1. Árbol de utilidad del sistema multi-agente (versión ampliada de la Figura 1)



Fuente: elaboración propia a partir de la matriz ASR.

Volver a la Figura 1 (apartado 3.3)

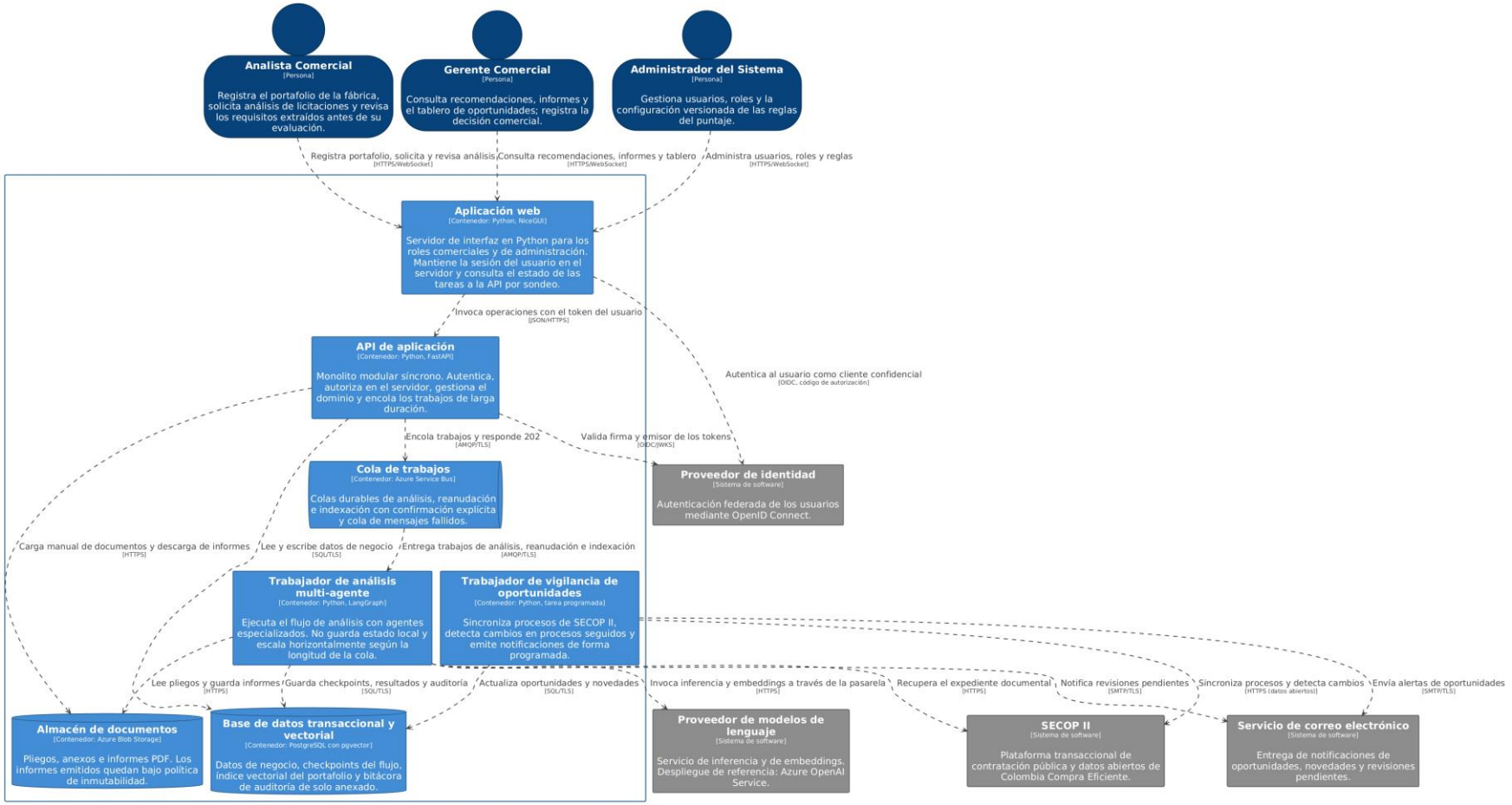
Figura C.2. Vista de contexto del sistema multi-agente (C4 Nivel 1) (versión ampliada de la Figura 2)



Fuente: elaboración propia con Structurizr.

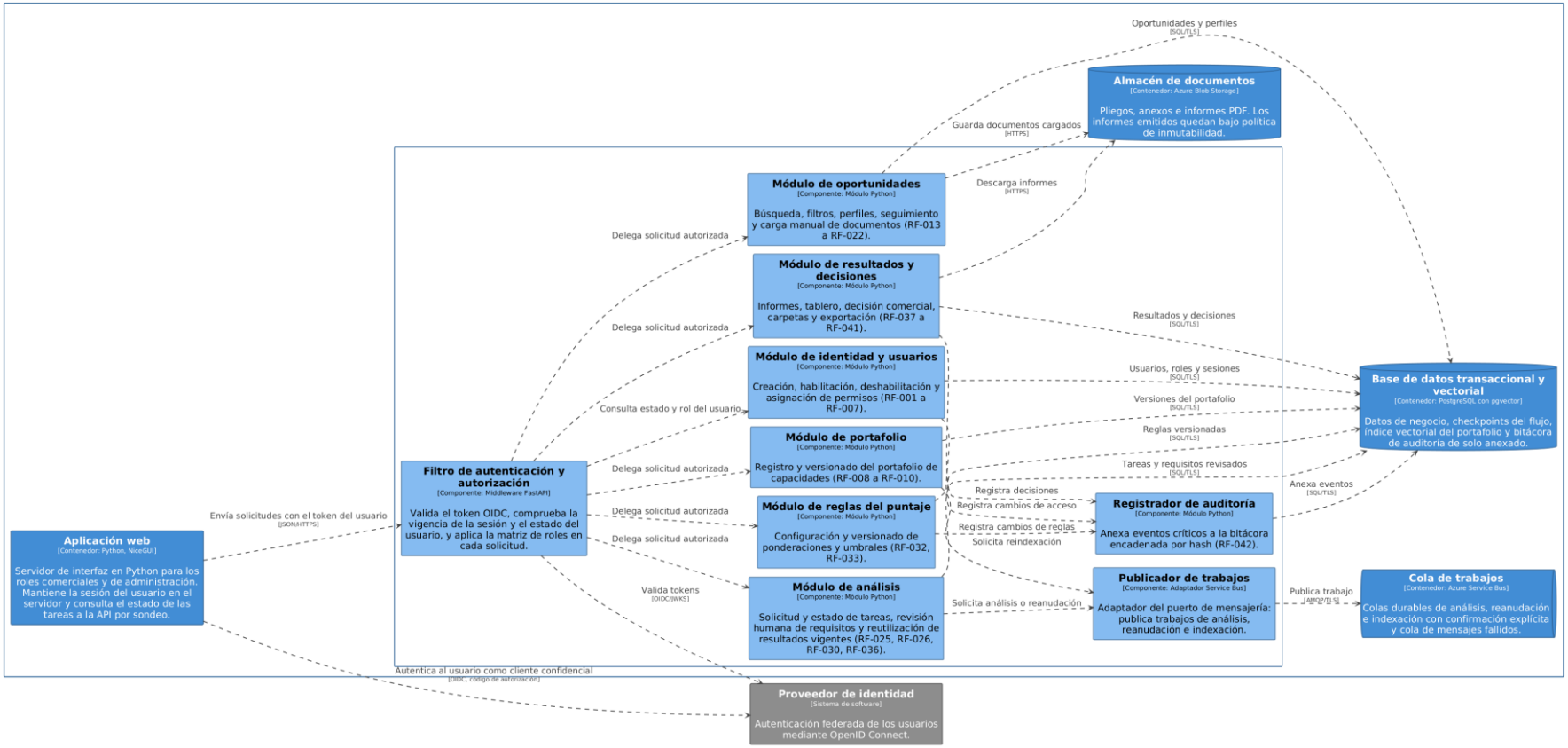
Volver a la Figura 2 (apartado 3.9)

Figura C.3. Vista de contenedores del sistema multi-agente (C4 Nivel 2) (versión ampliada de la Figura 3)



Fuente: elaboración propia con Structurizr.
Volver a la Figura 3 (apartado 3.10)

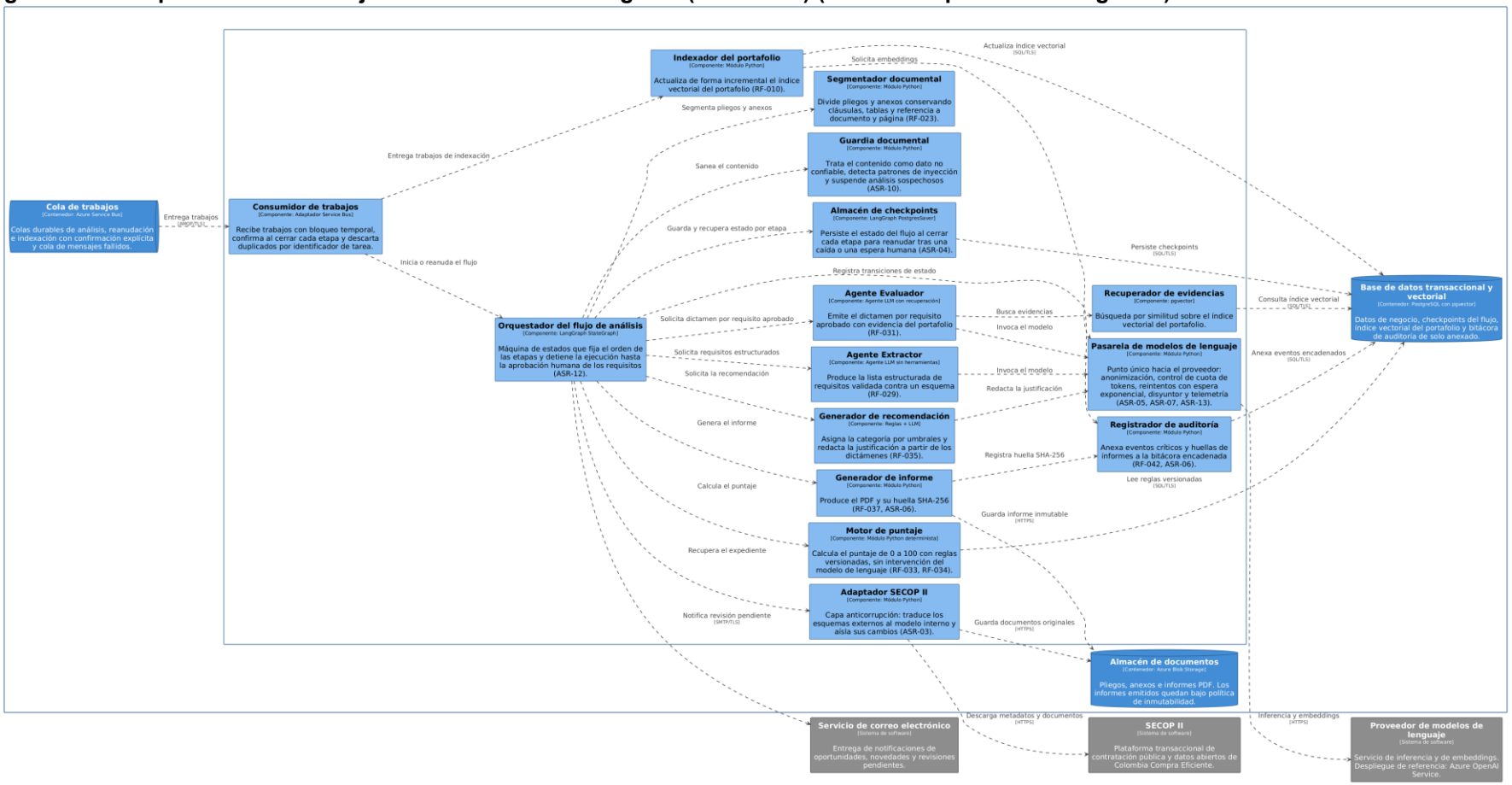
Figura C.4. Componentes de la API de aplicación (C4 Nivel 3) (versión ampliada de la Figura 4)



Fuente: elaboración propia con Structurizr.

Volver a la Figura 4 (apartado 3.11)

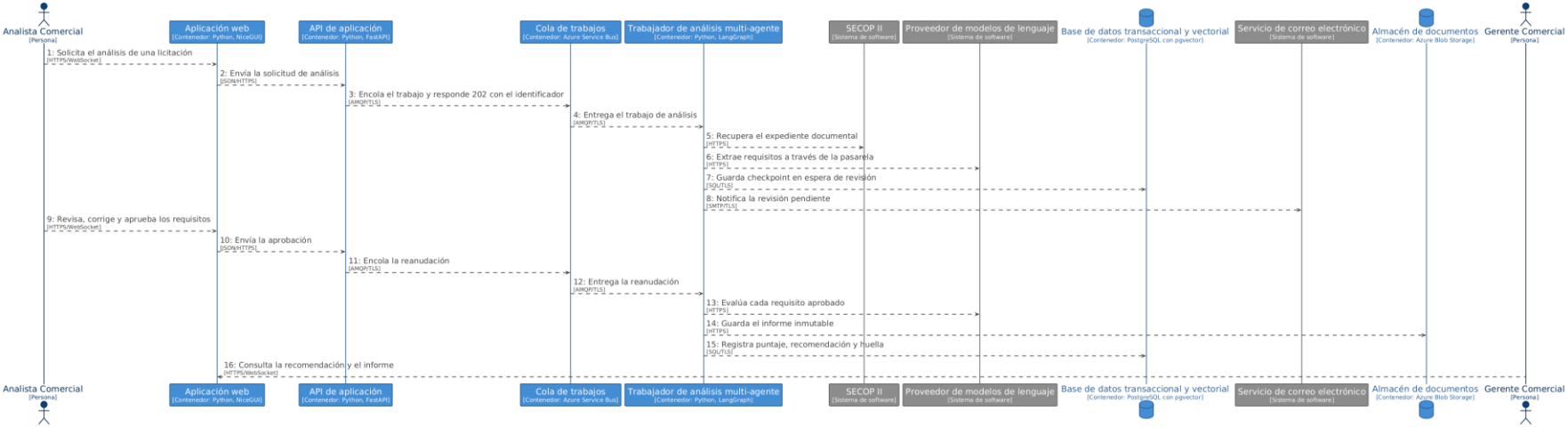
Figura C.5. Componentes del trabajador de análisis multi-agente (C4 Nivel 3) (versión ampliada de la Figura 5)



Fuente: elaboración propia con Structurizr.

[Volver a la Figura 5 (apartado 3.11)]

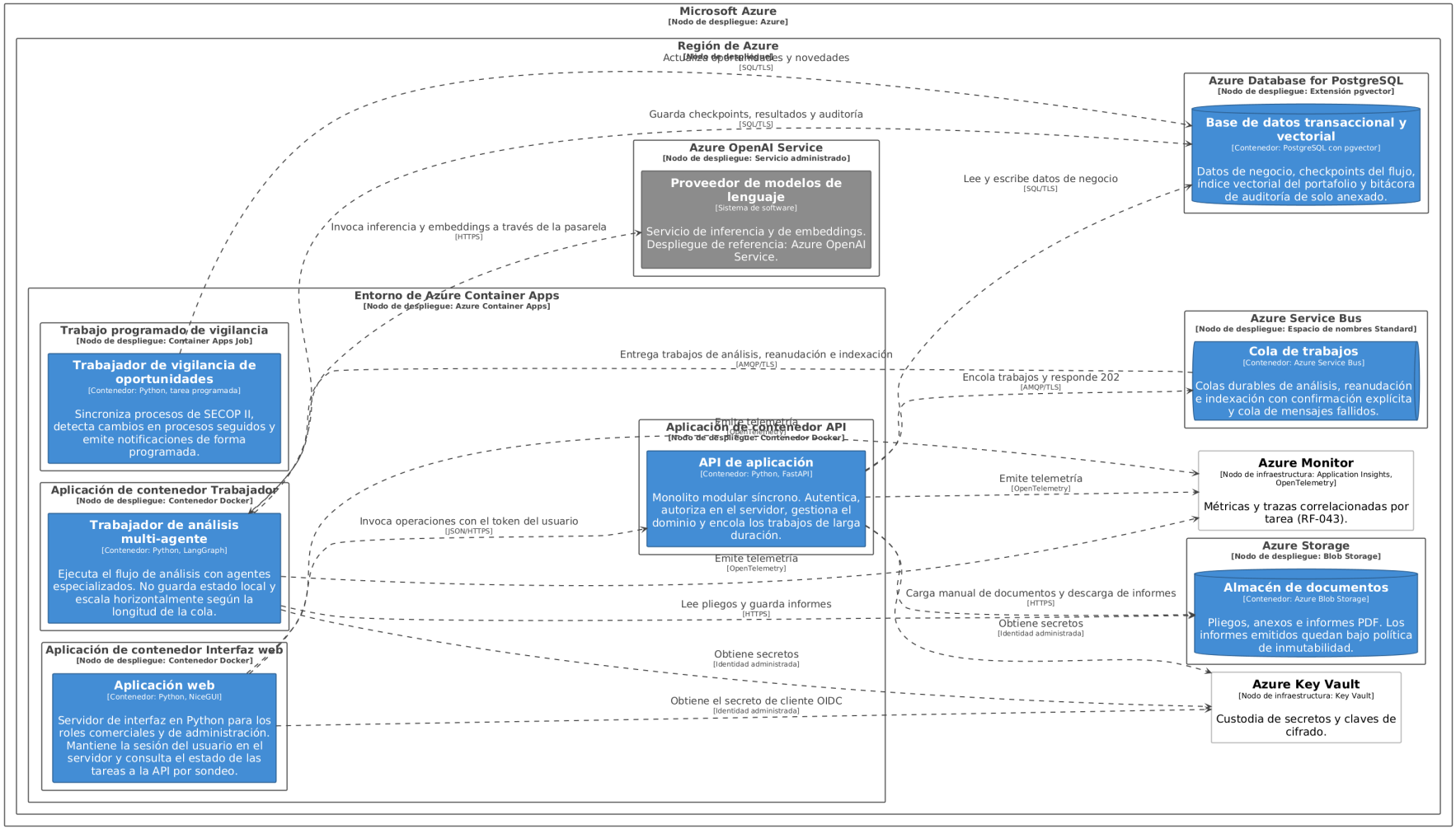
Figura C.6. Flujo de análisis con revisión humana en el ciclo (vista dinámica) (versión ampliada de la Figura 6)



Fuente: elaboración propia con Structurizr.

Volver a la Figura 6 (apartado 3.12)

Figura C.7. Despliegue de referencia del MVP en Azure (versión ampliada de la Figura 7)



Fuente: elaboración propia con Structurizr.

Volver a la Figura 7 (apartado 3.13)

Anexo D. Código fuente de los diagramas de arquitectura

Este anexo contiene el código con el que se generaron las figuras del capítulo 3. El espacio de trabajo de Structurizr (D.1) produce las Figuras 2 a 7 desde un único modelo; puede pegarse en el editor disponible en https://structurizr.com/dsl o abrirse con Structurizr Lite, y desde allí cada vista se exporta como imagen. El árbol de utilidad (D.2) está escrito en lenguaje DOT y se genera con Graphviz.

D.1 Espacio de trabajo Structurizr DSL (workspace.dsl)

```
workspace "Sistema multi-agente de análisis de licitaciones públicas" "Descripción arquitectónica del MVP conforme a
ISO/IEC/IEEE 42010:2022, expresada con el Modelo C4." {

    !impliedRelationships false

    model {

        # ---------------------------------------------------------------
        # Partes interesadas que operan el sistema (C4 Nivel 1)
        # ---------------------------------------------------------------
        analista = person "Analista Comercial" "Registra el portafolio de la fábrica, solicita análisis de licitaciones
y revisa los requisitos extraídos antes de su evaluación."
        gerente = person "Gerente Comercial" "Consulta recomendaciones, informes y el tablero de oportunidades; registra
la decisión comercial."
        admin = person "Administrador del Sistema" "Gestiona usuarios, roles y la configuración versionada de las reglas
del puntaje."

        # ---------------------------------------------------------------
        # Sistemas externos fuera del control del proyecto
        # ---------------------------------------------------------------
        secop = softwareSystem "SECOP II" "Plataforma transaccional de contratación pública y datos abiertos de Colombia
Compra Eficiente." "Externo"
        llm = softwareSystem "Proveedor de modelos de lenguaje" "Servicio de inferencia y de embeddings. Despliegue de
referencia: Azure OpenAI Service." "Externo"
        idp = softwareSystem "Proveedor de identidad" "Autenticación federada de los usuarios mediante OpenID Connect."
"Externo"
        correo = softwareSystem "Servicio de correo electrónico" "Entrega de notificaciones de oportunidades, novedades
y revisiones pendientes." "Externo"

        # ---------------------------------------------------------------
        # Entidad de interés (ISO/IEC/IEEE 42010:2022)
        # ---------------------------------------------------------------
        sma = softwareSystem "Sistema multi-agente de análisis de licitaciones" "Extrae los requisitos de los pliegos
publicados en SECOP II, evalúa su cumplimiento frente al portafolio de la fábrica de software y emite una recomendación
explicada." {

            webui = container "Aplicación web" "Servidor de interfaz en Python para los roles comerciales y de
administración. Mantiene la sesión del usuario en el servidor y consulta el estado de las tareas a la API por sondeo."
"Python, NiceGUI" "Interfaz" {
                perspectives {
                    "Seguridad" "Los tokens de acceso permanecen en el servidor de interfaz y no llegan al navegador; no
toma decisiones de autorización (ASR-09)."
                }
            }

            api = container "API de aplicación" "Monolito modular síncrono. Autentica, autoriza en el servidor, gestiona
el dominio y encola los trabajos de larga duración." "Python, FastAPI" {
                perspectives {
                    "Desempeño" "Responde 202 con identificador de tarea en menos de 1 s y delega el análisis a la cola
(ASR-11)."
                    "Seguridad" "Punto único de entrada para autenticación, autorización basada en roles y verificación
de sesión (ASR-09)."
                }
```

```
                filtroAuth = component "Filtro de autenticación y autorización" "Valida el token OIDC, comprueba la
vigencia de la sesión y el estado del usuario, y aplica la matriz de roles en cada solicitud." "Middleware FastAPI"
                modIdentidad = component "Módulo de identidad y usuarios" "Creación, habilitación, deshabilitación y
asignación de permisos (RF-001 a RF-007)." "Módulo Python"
                modPortafolio = component "Módulo de portafolio" "Registro y versionado del portafolio de capacidades
(RF-008 a RF-010)." "Módulo Python"
                modOportunidades = component "Módulo de oportunidades" "Búsqueda, filtros, perfiles, seguimiento y carga
manual de documentos (RF-013 a RF-022)." "Módulo Python"
                modAnalisis = component "Módulo de análisis" "Solicitud y estado de tareas, revisión humana de
requisitos y reutilización de resultados vigentes (RF-025, RF-026, RF-030, RF-036)." "Módulo Python"
                modReglas = component "Módulo de reglas del puntaje" "Configuración y versionado de ponderaciones y
umbrales (RF-032, RF-033)." "Módulo Python"
                modResultados = component "Módulo de resultados y decisiones" "Informes, tablero, decisión comercial,
carpetas y exportación (RF-037 a RF-041)." "Módulo Python"
                publicador = component "Publicador de trabajos" "Adaptador del puerto de mensajería: publica trabajos de
análisis, reanudación e indexación." "Adaptador Service Bus"
                auditoriaApi = component "Registrador de auditoría" "Anexa eventos críticos a la bitácora encadenada por
hash (RF-042)." "Módulo Python"
            }

            cola = container "Cola de trabajos" "Colas durables de análisis, reanudación e indexación con confirmación
explícita y cola de mensajes fallidos." "Azure Service Bus" "Cola" {
                perspectives {
                    "Fiabilidad" "Un trabajo no confirmado vuelve a la cola si el trabajador cae (ASR-04, ASR-07)."
                    "Escalabilidad" "La longitud de la cola dispara el escalado de trabajadores sin cambios de código
(ASR-08)."
                }
            }

            worker = container "Trabajador de análisis multi-agente" "Ejecuta el flujo de análisis con agentes
especializados. No guarda estado local y escala horizontalmente según la longitud de la cola." "Python, LangGraph" {
                perspectives {
                    "Desempeño" "Procesa en paralelo los fragmentos del pliego (ASR-01, ASR-02)."
                    "Fiabilidad" "Reanuda desde el último checkpoint persistido (ASR-04)."
                    "Escalabilidad" "Réplicas sin estado añadidas por la plataforma (ASR-08)."
                }

                consumidor = component "Consumidor de trabajos" "Recibe trabajos con bloqueo temporal, confirma al
cerrar cada etapa y descarta duplicados por identificador de tarea." "Adaptador Service Bus"
                orquestador = component "Orquestador del flujo de análisis" "Máquina de estados que fija el orden de las
etapas y detiene la ejecución hasta la aprobación humana de los requisitos (ASR-12)." "LangGraph StateGraph"
                checkpoints = component "Almacén de checkpoints" "Persiste el estado del flujo al cerrar cada etapa para
reanudar tras una caída o una espera humana (ASR-04)." "LangGraph PostgresSaver"
                adaptadorSecop = component "Adaptador SECOP II" "Capa anticorrupción: traduce los esquemas externos al
modelo interno y aísla sus cambios (ASR-03)." "Módulo Python"
                guardia = component "Guardia documental" "Trata el contenido como dato no confiable, detecta patrones de
inyección y suspende análisis sospechosos (ASR-10)." "Módulo Python"
                segmentador = component "Segmentador documental" "Divide pliegos y anexos conservando cláusulas, tablas
y referencia a documento y página (RF-023)." "Módulo Python"
                extractor = component "Agente Extractor" "Produce la lista estructurada de requisitos validada contra un
esquema (RF-029)." "Agente LLM sin herramientas"
                evaluador = component "Agente Evaluador" "Emite el dictamen por requisito aprobado con evidencia del
portafolio (RF-031)." "Agente LLM con recuperación"
                recuperador = component "Recuperador de evidencias" "Búsqueda por similitud sobre el índice vectorial
del portafolio." "pgvector"
                indexador = component "Indexador del portafolio" "Actualiza de forma incremental el índice vectorial del
portafolio (RF-010)." "Módulo Python"
                motorPuntaje = component "Motor de puntaje" "Calcula el puntaje de 0 a 100 con reglas versionadas, sin
intervención del modelo de lenguaje (RF-033, RF-034)." "Módulo Python determinista"
                recomendador = component "Generador de recomendación" "Asigna la categoría por umbrales y redacta la
justificación a partir de los dictámenes (RF-035)." "Reglas + LLM"
                generadorInforme = component "Generador de informe" "Produce el PDF y su huella SHA-256 (RF-037, ASR-
06)." "Módulo Python"
                pasarela = component "Pasarela de modelos de lenguaje" "Punto único hacia el proveedor: anonimización,
control de cuota de tokens, reintentos con espera exponencial, disyuntor y telemetría (ASR-05, ASR-07, ASR-13)." "Módulo
Python"
                auditoriaWorker = component "Registrador de auditoría" "Anexa eventos críticos y huellas de informes a
la bitácora encadenada (RF-042, ASR-06)." "Módulo Python"
            }
```

```
            vigilancia = container "Trabajador de vigilancia de oportunidades" "Sincroniza procesos de SECOP II, detecta
cambios en procesos seguidos y emite notificaciones de forma programada." "Python, tarea programada" {
                perspectives {
                    "Interoperabilidad" "Usa la misma capa anticorrupción que el trabajador de análisis (ASR-03)."
                    "Fiabilidad" "Una falla de SECOP II no afecta los análisis en curso (ASR-07)."
                }
            }


            bd = container "Base de datos transaccional y vectorial" "Datos de negocio, checkpoints del flujo, índice
vectorial del portafolio y bitácora de auditoría de solo anexado." "PostgreSQL con pgvector" "BaseDatos" {
                perspectives {
                    "Seguridad" "Cifrado en reposo con claves custodiadas; bitácora sin permisos de actualización ni
borrado (ASR-05, ASR-06)."
                    "Fiabilidad" "Respaldo continuo con restauración a un punto en el tiempo (ASR-04)."
                }
            }


            docs = container "Almacén de documentos" "Pliegos, anexos e informes PDF. Los informes emitidos quedan bajo
política de inmutabilidad." "Azure Blob Storage" "BaseDatos" {
                perspectives {
                    "Seguridad" "Informes inmutables verificables por huella SHA-256 (ASR-06)."
                }
            }
        }


        # ---------------------------------------------------------------------
        # Relaciones de Nivel 1 (contexto)
        # ---------------------------------------------------------------------
        analista -> sma "Registra el portafolio, solicita análisis y revisa requisitos"
        gerente -> sma "Consulta recomendaciones e informes y registra decisiones"
        admin -> sma "Administra usuarios, roles y reglas del puntaje"
        sma -> secop "Consulta procesos y descarga documentos" "HTTPS"
        sma -> llm "Solicita inferencia y embeddings" "HTTPS"
        sma -> idp "Delega la autenticación" "OpenID Connect"
        sma -> correo "Envía notificaciones" "SMTP/TLS"


        # ---------------------------------------------------------------------
        # Relaciones de Nivel 2 (contenedores)
        # ---------------------------------------------------------------------
        analista -> webui "Registra portafolio, solicita y revisa análisis" "HTTPS/WebSocket"
        gerente -> webui "Consulta recomendaciones, informes y tablero" "HTTPS/WebSocket"
        admin -> webui "Administra usuarios, roles y reglas" "HTTPS/WebSocket"
        webui -> idp "Autentica al usuario como cliente confidencial" "OIDC, código de autorización"
        webui -> api "Invoca operaciones con el token del usuario" "JSON/HTTPS"
        api -> idp "Valida firma y emisor de los tokens" "OIDC/JWKS"
        api -> bd "Lee y escribe datos de negocio" "SQL/TLS"
        api -> docs "Carga manual de documentos y descarga de informes" "HTTPS"
        api -> cola "Encola trabajos y responde 202" "AMQP/TLS"
        cola -> worker "Entrega trabajos de análisis, reanudación e indexación" "AMQP/TLS"
        worker -> secop "Recupera el expediente documental" "HTTPS"
        worker -> llm "Invoca inferencia y embeddings a través de la pasarela" "HTTPS"
        worker -> bd "Guarda checkpoints, resultados y auditoría" "SQL/TLS"
        worker -> docs "Lee pliegos y guarda informes" "HTTPS"
        worker -> correo "Notifica revisiones pendientes" "SMTP/TLS"
        vigilancia -> secop "Sincroniza procesos y detecta cambios" "HTTPS (datos abiertos)"
        vigilancia -> bd "Actualiza oportunidades y novedades" "SQL/TLS"
        vigilancia -> correo "Envía alertas de oportunidades" "SMTP/TLS"


        # ---------------------------------------------------------------------
        # Relaciones de Nivel 3: API de aplicación
        # ---------------------------------------------------------------------
        webui -> filtroAuth "Envía solicitudes con el token del usuario" "JSON/HTTPS"
        filtroAuth -> idp "Valida tokens" "OIDC/JWKS"
        filtroAuth -> modIdentidad "Consulta estado y rol del usuario"
        filtroAuth -> modPortafolio "Delega solicitud autorizada"
        filtroAuth -> modOportunidades "Delega solicitud autorizada"
        filtroAuth -> modAnalisis "Delega solicitud autorizada"
        filtroAuth -> modReglas "Delega solicitud autorizada"
        filtroAuth -> modResultados "Delega solicitud autorizada"
        modIdentidad -> bd "Usuarios, roles y sesiones" "SQL/TLS"
        modPortafolio -> bd "Versiones del portafolio" "SQL/TLS"
```

```
        modOportunidades -> bd "Oportunidades y perfiles" "SQL/TLS"
        modOportunidades -> docs "Guarda documentos cargados" "HTTPS"
        modAnalisis -> bd "Tareas y requisitos revisados" "SQL/TLS"
        modReglas -> bd "Reglas versionadas" "SQL/TLS"
        modResultados -> bd "Resultados y decisiones" "SQL/TLS"
        modResultados -> docs "Descarga informes" "HTTPS"
        modAnalisis -> publicador "Solicita análisis o reanudación"
        modPortafolio -> publicador "Solicita reindexación"
        publicador -> cola "Publica trabajo" "AMQP/TLS"
        modIdentidad -> auditoriaApi "Registra cambios de acceso"
        modReglas -> auditoriaApi "Registra cambios de reglas"
        modResultados -> auditoriaApi "Registra decisiones"
        auditoriaApi -> bd "Anexa eventos" "SQL/TLS"


        # ------------------------------------------------------------------
        # Relaciones de Nivel 3: Trabajador de análisis multi-agente
        # ------------------------------------------------------------------
        cola -> consumidor "Entrega trabajos" "AMQP/TLS"
        consumidor -> orquestador "Inicia o reanuda el flujo"
        consumidor -> indexador "Entrega trabajos de indexación"
        orquestador -> checkpoints "Guarda y recupera estado por etapa"
        checkpoints -> bd "Persiste checkpoints" "SQL/TLS"
        orquestador -> adaptadorSecop "Recupera el expediente"
        adaptadorSecop -> secop "Descarga metadatos y documentos" "HTTPS"
        adaptadorSecop -> docs "Guarda documentos originales" "HTTPS"
        orquestador -> guardia "Sanea el contenido"
        orquestador -> segmentador "Segmenta pliegos y anexos"
        orquestador -> extractor "Solicita requisitos estructurados"
        orquestador -> evaluador "Solicita dictamen por requisito aprobado"
        orquestador -> motorPuntaje "Calcula el puntaje"
        orquestador -> recomendador "Solicita la recomendación"
        orquestador -> generadorInforme "Genera el informe"
        orquestador -> auditoriaWorker "Registra transiciones de estado"
        orquestador -> correo "Notifica revisión pendiente" "SMTP/TLS"
        evaluador -> recuperador "Busca evidencias"
        recuperador -> bd "Consulta índice vectorial" "SQL/TLS"
        indexador -> pasarela "Solicita embeddings"
        indexador -> bd "Actualiza índice vectorial" "SQL/TLS"
        extractor -> pasarela "Invoca el modelo"
        evaluador -> pasarela "Invoca el modelo"
        recomendador -> pasarela "Redacta la justificación"
        pasarela -> llm "Inferencia y embeddings" "HTTPS"
        motorPuntaje -> bd "Lee reglas versionadas" "SQL/TLS"
        generadorInforme -> docs "Guarda informe inmutable" "HTTPS"
        generadorInforme -> auditoriaWorker "Registra huella SHA-256"
        auditoriaWorker -> bd "Anexa eventos encadenados" "SQL/TLS"


        # ------------------------------------------------------------------
        # Despliegue de referencia del MVP
        # ------------------------------------------------------------------
        mvp = deploymentEnvironment "MVP" {
            azure = deploymentNode "Microsoft Azure" "Suscripción dedicada a una fábrica de software (instancia de un
solo inquilino)." "Azure" {
                region = deploymentNode "Región de Azure" "Por definir según la política de transferencia internacional
de datos (Ley 1581 de 2012)." {
                    aca = deploymentNode "Entorno de Azure Container Apps" "Red virtual privada; escalado basado en
KEDA." "Azure Container Apps" {
                        webNode = deploymentNode "Aplicación de contenedor Interfaz web" "Afinidad de sesión activada."
"Contenedor Docker" {
                            webuiInst = containerInstance webui
                        }
                        apiNode = deploymentNode "Aplicación de contenedor API" "Mínimo una réplica activa." "Contenedor
Docker" {
                            apiInst = containerInstance api
                        }
                        workerNode = deploymentNode "Aplicación de contenedor Trabajador" "Réplicas de 0 a N según la
longitud de la cola." "Contenedor Docker" {
                            workerInst = containerInstance worker
                        }
                        vigNode = deploymentNode "Trabajo programado de vigilancia" "Ejecución periódica." "Container
Apps Job" {
                            vigInst = containerInstance vigilancia
                        }
```

```
                    }
                    sbNode = deploymentNode "Azure Service Bus" "Colas con cola de mensajes fallidos." "Espacio de
nombres Standard" {
                        colaInst = containerInstance cola
                    }
                    pgNode = deploymentNode "Azure Database for PostgreSQL" "Servidor flexible con respaldo continuo."
"Extensión pgvector" {
                        bdInst = containerInstance bd
                    }
                    stNode = deploymentNode "Azure Storage" "Contenedor de blobs con política de inmutabilidad para
informes." "Blob Storage" {
                        docsInst = containerInstance docs
                    }
                    aoai = deploymentNode "Azure OpenAI Service" "Despliegue de modelos en la misma suscripción."
"Servicio administrado" {
                        llmInst = softwareSystemInstance llm
                    }
                    kv = infrastructureNode "Azure Key Vault" "Custodia de secretos y claves de cifrado." "Key Vault"
                    monitor = infrastructureNode "Azure Monitor" "Métricas y trazas correlacionadas por tarea (RF-043)."
"Application Insights, OpenTelemetry"
                }
            }
            webuiInst -> kv "Obtiene el secreto de cliente OIDC" "Identidad administrada"
            webuiInst -> monitor "Emite telemetría" "OpenTelemetry"
            apiInst -> kv "Obtiene secretos" "Identidad administrada"
            workerInst -> kv "Obtiene secretos" "Identidad administrada"
            apiInst -> monitor "Emite telemetría" "OpenTelemetry"
            workerInst -> monitor "Emite telemetría" "OpenTelemetry"
        }
    }

    views {

        properties {
            "plantuml.title" "false"
        }

        terminology {
            person "Persona"
            softwareSystem "Sistema de software"
            container "Contenedor"
            component "Componente"
            deploymentNode "Nodo de despliegue"
            infrastructureNode "Nodo de infraestructura"
            relationship "Relación"
        }

        systemContext sma "N1_Contexto" "Figura: vista de contexto del sistema multi-agente (C4 Nivel 1)." {
            include *
            autoLayout tb 300 200
        }

        container sma "N2_Contenedores" "Figura: vista de contenedores (C4 Nivel 2)." {
            include *
            autoLayout tb 300 200
        }

        component api "N3_Componentes_API" "Figura: componentes de la API de aplicación (C4 Nivel 3)." {
            include *
            autoLayout lr 150 200
        }

        component worker "N3_Componentes_Trabajador" "Figura: componentes del trabajador de análisis multi-agente (C4
Nivel 3)." {
            include *
            autoLayout lr 150 200
        }

        dynamic sma "Dinamica_Analisis" "Figura: flujo de análisis con revisión humana en el ciclo." {
```

```
            analista -> webui "Solicita el análisis de una licitación"
            webui -> api "Envía la solicitud de análisis"
            api -> cola "Encola el trabajo y responde 202 con el identificador"
            cola -> worker "Entrega el trabajo de análisis"
            worker -> secop "Recupera el expediente documental"
            worker -> llm "Extrae requisitos a través de la pasarela"
            worker -> bd "Guarda checkpoint en espera de revisión"
            worker -> correo "Notifica la revisión pendiente"
            analista -> webui "Revisa, corrige y aprueba los requisitos"
            webui -> api "Envía la aprobación"
            api -> cola "Encola la reanudación"
            cola -> worker "Entrega la reanudación"
            worker -> llm "Evalúa cada requisito aprobado"
            worker -> docs "Guarda el informe inmutable"
            worker -> bd "Registra puntaje, recomendación y huella"
            gerente -> webui "Consulta la recomendación y el informe"
            properties {
                "plantuml.sequenceDiagram" "true"
            }
            autoLayout tb 200 150
        }

        deployment sma mvp "Despliegue_MVP" "Figura: despliegue de referencia del MVP en Azure." {
            include *
            autoLayout lr 250 150
        }

        styles {
            element "Element" {
                fontSize 22
            }
            element "Person" {
                shape Person
                background #08427b
                color #ffffff
            }
            element "Software System" {
                background #1168bd
                color #ffffff
            }
            element "Container" {
                background #438dd5
                color #ffffff
            }
            element "Component" {
                background #85bbf0
                color #000000
            }
            element "Externo" {
                background #8c8c8c
                color #ffffff
            }
            element "BaseDatos" {
                shape Cylinder
            }
            element "Cola" {
                shape Pipe
            }
            element "Infrastructure Node" {
                background #ffffff
                color #000000
            }
            relationship "Relationship" {
                fontSize 20
            }
        }
    }
}
```

D.2 Árbol de utilidad en Graphviz (figura1_arbol_utilidad.dot)

```
digraph UT {
  rankdir=LR; nodesep=0.18; ranksep=0.45;
  node [shape=box, style="rounded,filled", fontname="Arial", fontsize=11, penwidth=1];
  edge [color="#555555", arrowhead=none];
  root [label="Utilidad del\nsistema multi-agente", fillcolor="#08427b", fontcolor="white", fontsize=12];
  node [fillcolor="#1168bd", fontcolor="white"];
  ed [label="Eficiencia de\ndesempeño"]; co [label="Compatibilidad"]; fi [label="Fiabilidad"]; se [label="Seguridad"];
fl [label="Flexibilidad"];
  node [fillcolor="#438dd5", fontcolor="white"];
  r1 [label="Comportamiento\ntemporal"]; r2 [label="Capacidad"]; r3 [label="Interoperabilidad"]; r4
[label="Disponibilidad y\nrecuperabilidad"]; r7 [label="Tolerancia a fallos"]; r6 [label="Integridad y\nno repudio"]; r9
[label="Autenticidad"]; r8 [label="Escalabilidad"];
  node [fillcolor="white", fontcolor="black", shape=box, style="rounded"];
  a1 [label="ASR-01  p90 ≤ 10 min por pliego con 10 solicitudes concurrentes", color="#b30000", penwidth=2];
  a4 [label="ASR-04  RTO ≤ 4 h, RPO ≤ 12 h, 0 informes corruptos en simulacro", color="#b30000", penwidth=2];
  a7 [label="ASR-07  0 % de caídas del núcleo ante fallos 429/503 o timeout externo", color="#b30000", penwidth=2];
  a2 [label="ASR-02  p95: 10 evaluaciones concurrentes sin errores 5xx en 45 min", color="#d97a00", penwidth=2];
  a3 [label="ASR-03  ≥ 98 % de interpretación exitosa sobre 500 procesos", color="#d97a00", penwidth=2];
  a6 [label="ASR-06  100 % de informes verificables por huella SHA-256", color="#d97a00", penwidth=2];
  a8 [label="ASR-08  0 archivos modificados para duplicar capacidad en ≤ 5 min", color="#d97a00", penwidth=2];
  a9 [label="ASR-09  100 % de peticiones no autorizadas rechazadas en servidor", color="#6b6b6b", penwidth=2];
  node [shape=plaintext, style="", fontsize=10];
  p1 [label="(Alta, Alta)"]; p4 [label="(Alta, Alta)"]; p7 [label="(Alta, Alta)"]; p2 [label="(Alta, Media)"]; p3
[label="(Alta, Media)"]; p6 [label="(Alta, Media)"]; p8 [label="(Media, Alta)"]; p9 [label="(Media, Media)"];
  root -> {ed co fi se fl};
  ed -> r1 -> a1 -> p1; ed -> r2 -> a2 -> p2;
  co -> r3 -> a3 -> p3;
  fi -> r4 -> a4 -> p4; fi -> r7 -> a7 -> p7;
  se -> r6 -> a6 -> p6; se -> r9 -> a9 -> p9;
  fl -> r8 -> a8 -> p8;
  edge [style=invis];
}
```